Machine Learning for Tactile-to-Visual Terrain Learning

Undergraduate lecture notes for the ESA SSP Tactile LegWheel rig

From motor telemetry to terrain perception

Expanded edition: 25 October 2026

Abstract

This lesson develops a machine-learning programme for classifying sandy and rocky terrain from LegWheel motor and inertial telemetry, then transferring that experience into an RGB terrain predictor. It connects the repository's existing packet construction, contact timing, image projection, and spectral features to classical models, neural sequence models, weak supervision, Mamba-derived state-space models, and principal component analysis (PCA). Equations are included to make the internal mechanics of each method explicit. No implementation code is proposed; the emphasis is on intuition, experimental design, validation, and scientific interpretation.

How to study these notes. Read Sections 1–5 first, even if your main interest is Mamba. They explain the language in which later algorithms are written. Then study the signal representation and classical classifiers before the visual pipeline. The Mamba and PCA sections can be revisited independently. Work through the small examples with a calculator: reproducing one prediction or one gradient step is more useful than memorizing a model name. All numerical examples are illustrative and are not measurements from this rig.

The assumed background is school algebra, elementary differentiation, and a willingness to work with vectors. Linear algebra and probability are reviewed when needed. A paragraph headed *Check your understanding* is a short self-test; selected answers appear near the end.

Contents

1 Notation and mathematical language **3**

2 The objective **5**

3 What the repository already provides **7**

4 What constitutes one machine-learning example? **9**

5 Formal supervised-learning foundation **9**

6 Representing tactile data **13**

7 Suitable first-stage models **17**

8 Training and validation correctly **23**

9 Building the weakly supervised visual stage **26**

10 What *Where Should I Walk?* actually did **30**

11 Mamba-derived models **30**

12 Principal component analysis **36**

13 PCA at different FFT resolutions **39**

14 Code-specific data integrity issues **42**

15 Recommended research sequence **43**

16 A complete worked study design **43**

17 Exercises and selected answers **44**

18 Central lessons **47**

1 Notation and mathematical language

1.1 Examples, measurements, features, and labels

A machine-learning *example* is one input together with whatever target we want to predict. In this project, an example might be a telemetry window around one wheel–terrain interaction. A *measurement* is one sensor reading within that window. A *feature* is a number supplied to a classifier: it might be a raw reading, average torque, or power in a frequency band. A *label* is the desired answer. We use $y = 0$ for sand and $y = 1$ for rocks throughout the terrain-classification examples.

There are two distinct axes that are easy to confuse. The index i selects an example from the dataset; the index t selects a time sample within one example. Collecting 101 readings from one contact window does not create 101 independent labelled terrain interactions.

Symbol	Meaning and typical shape
N	Number of labelled examples in a dataset.
L	Number of time samples in one telemetry window.
C	Number of sensor channels; currently 15.
D	Number of scalar features supplied to a feature-based classifier.
$Z_i \in \mathbb{R}^{L\times C}$	Telemetry matrix for example i.
$\mathbf{r}_t \in \mathbb{R}^C$	Sensor vector at time t within a window.
$\mathbf{x}_i \in \mathbb{R}^D$	Feature vector summarizing example i.
$X \in \mathbb{R}^{N\times D}$	Dataset matrix; row i is $\mathbf{x}_i^\top$.
$y_i, \widehat{y}_i$	True class and predicted class for example i.
p_i	Model probability of rocks for example i.
$\mathbf{w}, b$	Learned weights and intercept of a linear score.
ϕ, θ	Collections of learned parameters; often tactile and visual.
$\ell_i, \mathcal{L}$	Loss on one example and combined training objective.
η, λ	Learning rate and regularization strength.
f_s, T_{win}	Sampling rate in Hz and physical window duration in seconds.
$\mathbf{h}_t$	Hidden state: a model's memory of earlier inputs.

Bold lowercase letters denote vectors in the explanations. Uppercase letters usually denote matrices. Some standard formulas use unbolded x for a scalar signal or, where explicitly stated, a vector. The joint angle q_h, the tactile distribution q_ϕ, and the PCA dimension q have different meanings; subscripts and the surrounding definition identify which is intended. In the Fourier formulas below, N_{FFT} denotes a signal length, distinct from dataset size N.

1.2 Vectors are lists; matrices are organized collections

Suppose the only features are mean wheel effort and vertical vibration RMS. One observation is

$$\mathbf{x}_i = \begin{bmatrix} 1.8 \\ 0.6 \end{bmatrix}.$$

This is a two-dimensional vector: a point in a plane whose axes are those two measurements. A classifier learns which regions of that plane correspond to each terrain. With 765 spectral features, the same picture applies in a 765-dimensional space, although we cannot draw all its axes.

The transpose symbol $^\top$ turns a column into a row. The dot product is a weighted sum:

$$\mathbf{w}^\top \mathbf{x} = \sum_{j=1}^{D} w_j x_j.$$

For $\mathbf{w} = (2, -1)^\top$, the example above gives $2(1.8) - 0.6 = 3.0$. The Euclidean norm $\|\mathbf{x}\|_2 = \sqrt{\sum_j x_j^2}$ measures length, and $\|\mathbf{x} - \mathbf{z}\|_2$ measures distance between two examples. For matrices, $\|X\|_F^2 = \sum_{i,j} X_{ij}^2$ is the sum of squared entries; the subscript F names the Frobenius norm.

A matrix product $A\mathbf{x}$ is a collection of dot products, one for each row of A. If A has shape $m \times D$, its output has m entries. This is how a neural layer creates several different weighted summaries from the same measurements. Checking shapes is an effective way to understand an equation and catch mistakes.

The identity matrix I leaves a vector unchanged. An inverse A^{-1}, when it exists, reverses the transformation A. The notation $\mathrm{diag}(s_1, \ldots, s_D)$ means a matrix with those numbers on its diagonal and zeros elsewhere. Multiplying by it scales each coordinate separately.

1.3 Probability describes uncertainty, not a different kind of class

The expression $P(Y = 1 \mid \mathbf{x})$ reads as the probability of rocks, given these features. The vertical bar means *conditioned on*. Uppercase Y emphasizes an uncertain quantity; y_i is one observed value. A prediction $p_i = 0.8$ still concerns a binary outcome. If the model is calibrated, about 80% of comparable predictions near 0.8 should be rocks. It does not necessarily mean that the patch is 80% covered by rocks.

For a random variable V, $\mathbb{E}[V]$ is its expected value, the average over repeated draws from its probability distribution. Its variance is $\mathrm{Var}(V) = \mathbb{E}[(V - \mathbb{E}[V])^2]$. Standard deviation is the square root of variance and therefore has the original measurement's units. For two variables, covariance is

$$\mathrm{Cov}(U, V) = \mathbb{E}[(U - \mathbb{E}[U])(V - \mathbb{E}[V])].$$

Positive covariance means they tend to rise together; negative covariance means they tend to move in opposite directions. Dividing covariance by both standard deviations gives correlation, a dimensionless number between -1 and 1. None of these quantities establishes causation.

The symbol $\mathcal{N}(\mu, \sigma^2)$ denotes a Gaussian or normal distribution with mean μ and variance σ^2. The expression $V \sim \mathcal{N}(\mu, \sigma^2)$ means that V is modeled as a draw from that distribution. A multivariate Gaussian replaces the variance by a covariance matrix Σ, describing an ellipsoidal cloud of feature vectors.

1.4 Optimization means improving parameters against a criterion

A model is a family of functions, such as $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. Training chooses parameters from that family. The notation

$$\phi^\star = \arg\min_{\phi} \mathcal{L}(\phi)$$

means the parameter choice giving the smallest objective. The minimum is a number; the argmin is the parameter setting that attains it. An asterisk denotes a selected or optimal value, and a hat commonly denotes an estimate or prediction.

The partial derivative $\partial \mathcal{L} / \partial w_j$ describes how the loss changes when weight w_j increases slightly. The gradient $\nabla_{\mathbf{w}} \mathcal{L}$ collects these derivatives into a vector. Moving against it decreases a smooth loss for a sufficiently small step:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}.$$

Here $\eta > 0$ is the learning rate. The arrow means assignment, not equality. The chain rule, $d\ell/dw = (d\ell/da)(da/dw)$, follows the effect of a weight through intermediate computations; backpropagation is its organized use in a network.

A *parameter* is learned from examples, such as a weight. A *hyperparameter* controls the learning procedure or model family, such as tree depth, regularization strength, or the number of PCA components. Hyperparameters are selected using validation data. A test set is reserved for assessing the chosen procedure.

Check your understanding. If X has 200 rows and 12 columns, what do the two dimensions mean? If $\mathbf{w} \in \mathbb{R}^{12}$, how many scores are in $X\mathbf{w}$? Why does a classifier's probability of rocks differ from a measurement of rock coverage?

2 The objective

2.1 What it means to learn from the rig

Imagine writing a rule that calls a surface rocky whenever vibration exceeds a chosen threshold. This is already a classifier. Machine learning replaces the hand-chosen rule by a rule whose parameters are estimated from examples. It does not remove the need to decide what an example means, which sensors to include, or how to judge success.

During *training*, the algorithm sees telemetry and labels together. During *inference*, it receives a new telemetry window and produces a prediction without seeing that window's label. We want it to recognize repeatable interaction patterns, rather than recognize a recording session. A useful learned rule should still work after the sand is repacked, the rocks are rearranged, or the experiment is repeated on another day.

There are three related tasks. *Classification* chooses a category, such as sand or rocks. *Regression* predicts a continuous quantity, such as slip or energy per metre. *Representation learning* constructs features useful for later tasks. PCA is an unsupervised representation method; a classifier trained using manual labels is supervised; using contact-derived targets to teach a camera is cross-modal learning.

2.2 A teacher that feels and a student that sees

The proposed system naturally separates into two learning problems. First, train a tactile or proprioceptive teacher

$$q_\phi(y \mid Z), \tag{2.1}$$

where $Z \in \mathbb{R}^{L \times C}$ is a contact-centred telemetry window, $C = 15$ for the existing packet representation, $y \in \{0, 1\}$ initially denotes sandy versus rocky terrain, and ϕ denotes learned parameters.

Second, transfer that experience into a visual student

$$p_\theta(y \mid I_{\text{patch}}), \tag{2.2}$$

where I_{patch} is the RGB region depicting the terrain that produced Z.

```
RGB image of a future terrain patch
                  |
                  | calibrated geometry and contact-time prediction
                  v
Telemetry when the wheel reaches that same patch
                  |
                  v
Tactile teacher: sandy/rocky probability
                  |
                  | soft pseudo-label
                  v
RGB student: predict terrain before touching it
                  |
                  v
Dense terrain-property or traversability map
```

Because the tactile teacher begins with manual sandy/rocky labels, the complete system is most accurately described as *cross-modal weak supervision* or teacher–student pseudo-labelling. It is not wholly self-supervised. A later version based on automatically measured slip, sinkage, vibration, or energy could be more properly called self-supervised.

If every packet has a manual sand/rock label, that same label can directly supervise its aligned RGB patch; a learned tactile teacher is not required for that subset. The teacher becomes valuable when it labels additional unannotated interactions, supplies calibrated soft targets, predicts continuous physical response, or transfers contact knowledge across environments.

There is also an important semantic distinction:

$$\text{terrain identity} \neq \text{terrain physical response.} \tag{2.3}$$

"Sandy" and "rocky" are material classes. "Loose" and "sturdy" are physical properties. Loose gravel is rocky but loose; compacted wet sand can be relatively sturdy. Ideally, retain both a semantic class and continuous physical targets such as slip, vibration, energy expenditure, or an empirical interaction score.

2.3 The prediction must match the intended decision

A wheel can identify a material only after interacting with it. A camera can observe it before contact. Transferring tactile experience to RGB therefore changes the time at which useful information becomes available. It does not make every physical property visible. Two surfaces can look similar but have different moisture or subsurface support; some uncertainty is irreducible from a single image.

For a terrain class, $p_\theta(y = 1 \mid I)$ estimates the probability of rocks. For a physical target, $\widehat{s}_\theta(I)$ might estimate slip under a specified speed and load. Those conditions matter. The same sand can be easy for one wheel and difficult for another. A practical response model may therefore condition on operating variables $\mathbf{u}$:

$$p_\theta(s \mid I, \mathbf{u}).$$

The symbol s here is a physical response, and $\mathbf{u}$ might contain commanded speed and load. This formulation makes explicit that traversability belongs to a robot–terrain interaction, not just to a material name.

Check your understanding. Why can a perfect sand/rock classifier still be an imperfect loose/sturdy predictor? Which extra measurements would make that distinction testable?

3 What the repository already provides

The raw MCAP contains motor states, encoder data, RGB, depth, IMU, requested torque, control settings, and transforms. The offline `legwheel_dataset` package then produces contact-centred packets. Its high-level design is documented in `src/legwheel_dataset/README.md`.

3.1 Telemetry channels

The packet reader constructs 15 channels in this order. One time sample, written as a row for compactness, is

$$\mathbf{r}_t^\top = \begin{bmatrix} q_h & \dot{q}_h & \tau_h & q_k & \dot{q}_k & \tau_k & q_w & \dot{q}_w & \tau_w & \omega_x & \omega_y & \omega_z & a_x & a_y & a_z \end{bmatrix}. \tag{3.1}$$

These are hip, knee, and wheel position, velocity, and effort, followed by three-axis IMU angular velocity and linear acceleration. The exact order is defined in `legwheel_dataset/bag_reader.py`.

The derived packets therefore contain nine motor/joint channels and six IMU channels. They do not currently contain motor current, voltage, temperature, or fault state. Effort is the motor torque estimate in newton metres. The central rotation encoder is used to correct the rig trajectory and determine contact time, but it is deliberately excluded from the ML arrays and FFTs.

Reading the physical symbols. The letter q denotes a joint angle, $\dot{q} = dq/dt$ its angular velocity, and τ its effort or torque estimate. The subscripts h, k, w select hip, knee, and wheel. Angular positions are in radians, angular velocities in radians per second, and torque in newton metres. The IMU, or inertial measurement unit, reports angular velocity $\omega_x, \omega_y, \omega_z$ and linear acceleration a_x, a_y, a_z. These measurements describe motion and loading of the rig; they do not directly identify the terrain.

Stacking the sample rows gives Z_i. At the default spectral length, it has shape 101×15: 101 moments, each containing 15 sensor values. Flattening it would give 1,515 inputs, but would hide its time/channel structure from an ordinary linear layer. A temporal network preserves that structure by treating neighbouring rows as neighbouring times.

3.2 Two time windows

Each `telemetry.npz` stores a local, contact-patch window and a fixed spectral window, together with cell-wise validity masks and times relative to predicted contact. Both windows are centred on contact rather than on camera exposure.

The local duration is

$$T_{\text{local}} = \frac{\alpha_{\text{patch}}}{\overline{|\dot{\theta}_{\text{rig}}|}}, \tag{3.2}$$

so it approximates the time required for the wheel to traverse the selected patch. Because rig speed varies by run, local arrays can have different lengths between runs. The spectral window defaults to one second at 100 Hz and is therefore more convenient for a fixed-size classifier.

Worked example: why window length depends on speed. If a selected patch spans $\alpha_{\text{patch}} = 0.10$ rad and the rig moves at an average angular speed of 0.20 rad/s, its traversal time is $0.10/0.20 = 0.50$ s. At 0.10 rad/s, the same patch requires 1 s. A local window can therefore follow the physical patch, whereas a fixed one-second window follows a fixed amount of time.

Neither choice is universally correct: one controls spatial context, the other temporal context. Comparing them asks a meaningful scientific question.

The validity mask is as important as the values. A zero measurement can be real; an absent measurement is unknown. If a channel contains a missing interval, silently filling it with zeros can manufacture an apparent impact and create broadband FFT power. The mask allows the pipeline to distinguish these cases. Interpolation fills small gaps under a smoothness assumption; it does not recover a genuinely unobserved collision.

3.3 Tactile-to-RGB correspondence

For an image acquired at t_{image}, the processor chooses a physical annular-sector patch ahead of the wheel:

$$\theta_{\text{patch}} = \theta_y(t_{\text{image}}) + d\,\alpha_{\text{off}}, \tag{3.3}$$

where $d \in \{-1, +1\}$ is the motion direction and α_{off} defaults to 0.35 rad. It then solves the future contact condition

$$\theta_y(t_{\text{contact}}) = \theta_{\text{patch}}. \tag{3.4}$$

Telemetry windows are centred at the predicted t_{contact}, while RGB and depth masks are intended to identify the same physical patch at their respective image timestamps. The repository implements the nominal geometric and temporal association machinery, a central asset for this project. Its correspondence is not established as correct until the conflicting projection constants and cross-stream clock offsets and drift have been physically validated, as discussed in the data-integrity section.

Worked example: the relevant image is earlier than the contact. At nearly constant rig speed 0.20 rad/s and angular offset 0.35 rad, the wheel arrives approximately $0.35/0.20 = 1.75$ s after the image. Pairing that image with telemetry at image exposure would describe a different patch. The geometric contact-time calculation supplies the required delay. The actual processor uses the trajectory rather than assuming constant speed. This distinction is central to creating meaningful visual labels.

3.4 Existing spectral products

For each of the 15 channels, Stage 2 checks data coverage, fills only short internal gaps, removes a linear trend, applies a Hann window, computes a one-sided power spectral density (PSD), and stores both the complete spectrum and the two strongest non-DC local peaks. The implementation is in `legwheel_dataset/features.py`.

At the defaults, one packet contains a 51×15 spectral-power array, giving 765 candidate spectral features. No recorded MCAP or generated packet dataset was available for the original source review, so numerical signal ranges, dropout rates, class balance, and spectral separability in these notes remain unmeasured. The examples are hypotheses and teaching illustrations.

The storage stages have different jobs. MCAP is the recorded message container. A packet aligns a physical image patch with its later interaction. The file `telemetry.npz` retains sampled sensor arrays. The file `features.npz` contains derived products, including `spectral_power`. A learning dataset would additionally associate each packet with its manual label and experiment group. Files that are conveniently aligned are not automatically ready for valid statistical evaluation.

4 What constitutes one machine-learning example?

A packet is the obvious storage unit, but it is not necessarily an independent statistical example. With packets saved at up to 10 Hz and a one-second FFT window, adjacent windows can overlap by approximately 90%. Two neighbouring packets may therefore contain almost the same telemetry and nearly identical images.

If neighbouring packets are randomly split between training and testing, a model can appear excellent while merely memorizing a particular run or patch. The correct hierarchy is closer to:

```
terrain material batch
‘-- independently prepared physical terrain setup
    ‘-- recording session or day
        ‘-- run
            ‘-- lap
                ‘-- overlapping packets
```

The train/test split should occur as high in that hierarchy as practical. A strong test set holds out an independently prepared sand or rock bed, a separate recording session, different terrain placement around the rig, and ideally different moisture, compaction, lighting, and rock arrangement. All laps, packets, and FFT resolutions derived from one raw run must remain in the same split.

Worked example: packet count versus experimental diversity. Suppose there are 10 runs with 1,000 overlapping packets each. A random 80/20 packet split places about 800 training and 200 test packets from every run on both sides. It asks whether the model can interpolate within known runs. Holding out two complete runs asks a harder question. Holding out a new material preparation and day asks a harder and usually more useful question again. These are different claims, even if all use the word *accuracy*.

Overlapping packets are still useful. They provide more views, reduce dependence on a particular contact alignment, and help train a temporal model. Their number should not be confused with the number of independent experimental replications. Uncertainty estimates should resample the latter.

Avoid imposing a nominal five-fold split when there are too few independent groups to populate every fold with both classes. A smaller defensible evaluation is better than a larger nominal fold count that compares one class or one session against itself.

5 Formal supervised-learning foundation

5.1 A complete learning procedure

It helps to separate three questions. The *representation* decides which numbers describe the interaction. The *model* decides how those numbers can be combined. The *training objective* decides what makes one parameter choice better than another. For example, log-band powers, a linear score, and cross-entropy are three independent design choices.

For a feature-based model, write $\mathbf{x}_i = \psi(Z_i)$, where ψ is the feature extractor. An end-to-end neural model learns much of ψ together with the classifier. In either case, training is a search for a rule that performs well on new observations. It is not a search for a rule that merely explains all training examples.

Let the manually labelled dataset be

$$\mathcal{D} = \{(Z_i, y_i, g_i)\}_{i=1}^{N}, \tag{5.1}$$

where g_i identifies a physical setup or recording group. A model f_ϕ normally minimizes empirical risk:

$$\phi^\star = \arg\min_{\phi} \frac{1}{N} \sum_{i=1}^{N} w_i \, \ell(f_\phi(Z_i), y_i) + \lambda R(\phi). \tag{5.2}$$

Here ℓ is the loss, w_i can compensate for class imbalance or label confidence, R is a regularizer, and λ controls model complexity.

Weights also change what the fitted probabilities mean. With positive class weights β_0, β_1, the population optimum of weighted cross-entropy is

$$p_{\text{weighted}}(\mathbf{x}) = \frac{\beta_1 p_0(\mathbf{x})}{\beta_1 p_0(\mathbf{x}) + \beta_0[1 - p_0(\mathbf{x})]},$$

where p_0 is the probability under the unweighted training distribution. This corresponds to a changed effective class prior. For example, $p_0 = 0.2$ and weights $(\beta_0, \beta_1) = (1, 4)$ give $p_{\text{weighted}} = 0.5$. Raw weighted outputs are therefore not automatically deployment probabilities. Use an appropriate prior correction under its assumptions, or calibrate on data representative of deployment prevalence; then assess calibration on independent groups.

The real objective is expected loss on a new terrain setup:

$$\mathcal{R}(\phi) = \mathbb{E}_{(Z,y)\sim p_{\text{future}}} \left[\ell(f_\phi(Z), y)\right]. \tag{5.3}$$

This distinction is why evaluation design matters more than packet count.

5.2 Underfitting, overfitting, and regularization

An overly rigid model can *underfit*: one threshold may be unable to separate surfaces whose distinction depends jointly on effort and vibration. An overly flexible model can *overfit*: it can memorize a distinctive sequence from each training run. Training error then falls while validation error stops improving or increases.

The *bias–variance trade-off* describes this tension. Bias is systematic error caused by restrictive assumptions; variance is sensitivity to which training examples happened to be collected. More flexibility can reduce bias while increasing variance. More independent training environments and regularization can reduce variance, although no formula replaces checking held-out performance.

For intuition, imagine fitting a curve through noisy effort measurements. A straight line may miss a genuine bend; a very wiggly curve may pass through every noise fluctuation. The useful curve captures structure that survives repetition. Terrain classification has the same problem in a higher-dimensional feature space.

5.3 Binary cross-entropy

For binary classification, a logistic output is

$$p_i = P(y_i = 1 \mid Z_i) = \sigma(s_i) = \frac{1}{1 + e^{-s_i}}. \tag{5.4}$$

A Bernoulli label has likelihood

$$P(y_i \mid p_i) = p_i^{y_i}(1 - p_i)^{1-y_i}. \tag{5.5}$$

Maximizing the product of these likelihoods is equivalent to minimizing negative log-likelihood:

$$\mathcal{L}_{\mathrm{BCE}} = -\sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \tag{5.6}$$

The formula packages two cases. If $y_i = 1$, the loss is $-\log p_i$. If $y_i = 0$, it is $-\log(1 - p_i)$. A correct confident prediction has a small loss. A confidently wrong prediction has a large loss because the logarithm diverges as the assigned probability of the observed class goes to zero. Natural logarithms are used throughout.

True class y_i	Predicted rock probability	Probability of true class	Loss
1	0.9	0.9	0.105
1	0.5	0.5	0.693
1	0.1	0.1	2.303
0	0.9	0.1	2.303

Why multiply likelihoods and then take a logarithm? Under a working conditional-independence assumption, the probability assigned to all labels is the product of the individual probabilities. A product of many small numbers is awkward to compute. A logarithm turns it into a sum and preserves which parameter choice is best. Negating changes maximization into minimization. For overlapping packets this independence assumption is an approximation; we can still optimize a sum of losses, but must evaluate and weight the correlated observations appropriately.

For a linear score $s_i = w^\top x_i + b$,

$$\frac{\partial \ell_i}{\partial w} = (p_i - y_i) x_i. \tag{5.7}$$

If the prediction is correct and confident, $p_i - y_i$ is near zero. If it is confidently wrong, the update is large and directed along the features active in that example.

5.4 Deriving and using the gradient

For one example, let $a = \mathbf{w}^\top \mathbf{x} + b$ be the score and $p = \sigma(a)$ the probability. The derivative of the sigmoid is $dp/da = p(1 - p)$. Differentiating the loss with respect to p gives

$$\frac{d\ell}{dp} = -\frac{y}{p} + \frac{1 - y}{1 - p}.$$

Multiply by dp/da, cancel terms, and obtain

$$\frac{d\ell}{da} = p - y, \qquad \nabla_{\mathbf{w}} \ell = (p - y)\mathbf{x}, \qquad \frac{d\ell}{db} = p - y.$$

This is why logistic training is intuitive: the probability error $p - y$ scales each active feature. If rocks were underpredicted, the score is pushed up; if rocks were overpredicted, it is pushed down.

Worked example: one update. Take $\mathbf{x} = (1, 2)^\top$, $y = 1$, and initially $\mathbf{w} = (0, 0)^\top, b = 0$. Then $a = 0$, $p = 0.5$, and the weight gradient is $(-0.5, -1)^\top$. With learning rate $\eta = 0.1$, the new weights are $(0.05, 0.10)^\top$, and $b = 0.05$. The same example now has score $0.05 + 0.20 + 0.05 = 0.30$ and $p \approx 0.574$. A single step has moved the prediction toward its label. Other examples will push in other directions, so training balances them.

A *batch* contains several examples whose gradients are averaged before the update. An *epoch* is one pass through the training dataset. Stochastic or mini-batch gradient descent uses subsets, making updates cheaper but noisier. Too large a learning rate can overshoot; too small a rate can make progress slow. Adam adapts update scales using running statistics of gradients, but it still needs a validation-controlled learning rate and training duration.

5.5 Regularization

For logistic regression, L2 regularization gives

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE}} + \lambda \|w\|_2^2. \tag{5.8}$$

It discourages large, unstable coefficients and is especially valuable when spectral bins are strongly correlated. L1 regularization,

$$\lambda \|w\|_1, \tag{5.9}$$

can set many coefficients to zero. It is useful for exploratory frequency selection, although correlated neighbouring bins can lead to unstable choices.

L2 adds $2\lambda \mathbf{w}$ to the gradient when the penalty is $\lambda \|\mathbf{w}\|_2^2$. Even without a classification error, weights are pulled toward zero. In a probabilistic interpretation this corresponds to preferring weights near zero under a Gaussian prior. L1 produces a piecewise constant pull and can place an optimum exactly at zero; this is why it encourages sparse solutions.

The scale of the loss matters. A summed data loss grows with dataset size; an averaged loss does not. Consequently the same numerical λ can have different effects under these conventions. State the convention when comparing experiments. The intercept is usually left unpenalized.

Regularization is broader than weight penalties. Limiting tree depth, using a small network, early stopping, and collecting varied training setups all constrain memorization in different ways. *Early stopping* chooses a training checkpoint using validation loss; the final test set must not choose that checkpoint.

5.6 Decision thresholds and safety

The default decision $p > 0.5$ is optimal only when the two error types have equal cost, with calibrated probabilities for the deployment population. For this separate decision example, define $U = 1$ to mean unsafe or very loose terrain and let $p = P(U = 1 \mid x)$. This is different from assigning rocks the label 1: a rocky surface is not necessarily unsafe. Let C_{FP} be the cost of unnecessarily avoiding safe terrain and C_{FN} the cost of treating unsafe terrain as safe. The minimum-cost threshold is

$$p > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}. \tag{5.10}$$

When a false-safe decision is dangerous, C_{FN} is large and the threshold becomes lower. Calibrated probabilities therefore matter more than accuracy alone for navigation.

The derivation compares expected costs. Predicting unsafe costs $C_{\text{FP}}(1-p)$; predicting safe costs $C_{\text{FN}}p$. Choose unsafe when the former is smaller. Rearranging gives the threshold above. With costs 1 and 9, the threshold is 0.1. A terrain-class probability can inform a planning decision only after specifying how class, physical response, and navigation cost relate.

Check your understanding. For a rock example with $p = 0.2$, what is the sign of $d\ell/da$? Why does subtracting that gradient increase the score? Why might a lower training loss coincide with a worse test result?

6 Representing tactile data

6.1 Why motor and inertial data can contain terrain information

The motor does not sense a material label. It participates in a mechanical interaction whose response depends on the surface. A simplified wheel balance is

$$J\dot{\omega} = \tau_{\text{motor}} - \tau_{\text{contact}} - \tau_{\text{loss}},$$

where J is effective rotational inertia, ω angular velocity, τ_{contact} terrain reaction torque, and τ_{loss} other losses. A change in reaction torque must affect acceleration, motor torque, or both. The controller decides how that change appears.

Under approximately constant torque, a harder-to-drive surface may reduce speed. Under approximately constant speed, the controller may increase torque to compensate. Thus a rule learned under one control mode can fail under another, even if both record the same telemetry fields. Commanded torque and measured effort are also different quantities; logging both does not make them interchangeable.

A simple contact analogy is a spring and damper:

$$F = k\delta + c\dot{\delta},$$

with compression δ, stiffness k, damping coefficient c, and reaction force F. A sharp rise in support can excite vibrations; a deformable bed can dissipate energy through displacement and rearrangement. Real sand, gravel, and rocks are nonlinear, history-dependent, and often involve slip, so this equation is an intuition aid rather than a complete terrain model.

The IMU records some resulting body motion, filtered by wheel compliance, joint control, sensor mounting, and the rig structure. A strong spectral peak may be a structural resonance excited by several surfaces. Its amplitude or relation to effort may still be useful, but the peak itself is not proof of a material property. This motivates testing at multiple operating conditions and looking for repeatable relationships rather than naming terrain from a single frequency.

6.2 A representation decides which differences survive

Two windows can differ in overall torque, vibration strength, impact timing, or coordination between joints. A representation preserves some of these differences and removes others. A mean discards ordering. A spectrum retains frequency content but loses absolute phase. A raw sequence keeps ordering but asks the learning algorithm to discover useful summaries. Representation is therefore a scientific assumption about what should matter.

For example, the sequences $(0, 0, 4, 0)$ and $(4, 0, 0, 0)$ have the same mean and RMS, yet the impact occurs at a different time relative to contact. If that timing indicates a rock edge, summary features lose useful evidence. If the alignment is noisy and timing is irrelevant, discarding it can improve robustness. More information is not automatically easier to learn from.

6.3 Hand-engineered summary features

For every channel, useful time-domain candidates include mean, median, standard deviation, RMS, median absolute deviation, extrema, peak-to-peak range, skewness, kurtosis, derivative statistics, impulse and crest factors, and cross-channel correlations.

For one channel with readings $v_1, \ldots, v_L$, useful definitions are

$$\bar{v} = \frac{1}{L}\sum_{t=1}^{L} v_t, \qquad s_v^2 = \frac{1}{L-1}\sum_{t=1}^{L}(v_t - \bar{v})^2, \tag{6.1}$$

$$\mathrm{RMS}(v) = \sqrt{\frac{1}{L}\sum_{t} v_t^2}, \qquad \mathrm{range}(v) = \max_t v_t - \min_t v_t. \tag{6.2}$$

The mean describes the baseline load. Standard deviation describes variation around that baseline. RMS includes both baseline and fluctuation, so it should not be treated as synonymous with vibration after mean removal. For the window variance defined with divisor L, $\mathrm{RMS}^2 = \bar{v}^2 + \mathrm{Var}_L(v)$.

An impact can be conspicuous even when the mean is small. Kurtosis, approximately $\frac{1}{L}\sum_t[(v_t - \bar{v})/s_v]^4$, strongly weights large deviations; conventions differ on the variance divisor and on subtracting 3. It can indicate impulsiveness but is sensitive to outliers. Median absolute deviation, $\mathrm{median}_t\,|v_t - \mathrm{median}(v)|$, is more robust to isolated extremes. The derivative approximation $(v_t - v_{t-1})f_s$ measures rapid change but amplifies measurement noise.

Absolute position deserves special treatment. The average wheel angle can identify track sector or lap rather than terrain. Relative-to-contact positions, angular increments, or omitting position are often better first experiments. Angles that wrap at 2π need unwrapping or a circular representation; averaging values just below 2π and just above zero as ordinary numbers gives a misleading result.

Motor mechanical power and an energy proxy are

$$P(t) = \tau(t)\dot{q}(t), \qquad E = \int |\tau(t)\dot{q}(t)|\,dt. \tag{6.3}$$

Physics suggests, but does not guarantee, that rocks may produce short impacts, high kurtosis, and broadband vibration, while sand may produce smoother high-frequency response but greater low-frequency effort or slip. Loose stones may combine both. Control settings and speed can reverse these tendencies, so they must be treated as hypotheses rather than assumed truths.

6.4 Standardization and missing values

Suppose effort varies by 0.2 N m while an acceleration feature varies by 5 $\mathrm{m/s^2}$. Euclidean distances and regularization penalties depend on these scales. Standardization replaces feature j by

$$x'_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

where μ_j and s_j are estimated from training examples only. One standardized unit means one training standard deviation. A constant feature has $s_j = 0$ and should be removed or handled explicitly.

Changing units should not change which physical differences a model regards as important. Scaling helps achieve that for linear penalties, distances, SVM kernels, and PCA. Trees depend mainly on feature order, so ordinary positive rescaling has little effect on their splits.

A simple missing-value strategy replaces a feature by its training median and adds an indicator marking that it was missing. The indicator can help, but can also expose terrain-dependent sensor failure. Report performance with and without such indicators. Imputation cannot be fitted using the test set, and extensive missingness may justify excluding a window rather than manufacturing a plausible complete signal.

6.5 Frequency-domain features

Think of a signal as a mixture of slow and fast oscillations. The discrete Fourier transform, or DFT, asks how strongly it matches each oscillatory template. The fast Fourier transform, or FFT, is an efficient algorithm for computing the DFT; it is not a different representation.

For samples $x[n]$, $n = 0, \ldots, N_{\mathrm{FFT}} - 1$, the DFT is

$$F[k] = \sum_{n=0}^{N_{\mathrm{FFT}}-1} x[n] e^{-\mathrm{i}2\pi kn/N_{\mathrm{FFT}}}. \tag{6.4}$$

Here $\mathrm{i}^2 = -1$ is the imaginary unit, distinct from the example index i. Euler's identity $e^{-\mathrm{i}\alpha} = \cos\alpha - \mathrm{i}\sin\alpha$ shows that the transform compares the data with both cosine and sine templates. The magnitude $|F[k]|$ is an unnormalized spectral-coefficient magnitude, only proportional to sinusoidal amplitude under suitable conditions. Recovering amplitude depends on DFT normalization, alignment with a frequency bin, one-sided scaling, and the analysis window's coherent gain. For the unwindowed DFT above, an isolated, bin-aligned real cosine of amplitude A has $|F[k]| = N_{\mathrm{FFT}}A/2$ away from DC and Nyquist. With a window, its coherent gain $G_c = N_{\mathrm{FFT}}^{-1} \sum_n w[n]$ is an additional amplitude scaling factor; off-bin leakage also needs consideration. The complex angle contains phase, or where in the oscillation the signal starts. The corresponding bin frequency and spacing are

$$f_k = \frac{kf_s}{N_{\mathrm{FFT}}}, \qquad \Delta f = \frac{f_s}{N_{\mathrm{FFT}}}. \tag{6.5}$$

Worked example: recognizing a frequency. For $f_s = 100$ Hz and $N_{\mathrm{FFT}} = 100$, a 10 Hz cosine completes ten cycles in the nominal one-second DFT interval. It aligns with bin $k = 10$. Its positive and negative frequency components are paired because the input is real. A real FFT stores only the nonnegative half. A time shift changes the pair's phases but not their magnitudes. This explains both the robustness and the information loss of power features. For a unit-amplitude cosine in this unwindowed example, each corresponding DFT coefficient has magnitude 50; one-sided amplitude recovery gives $2(50)/100 = 1$.

Sampling imposes a limit. Frequencies above $f_s/2$, the Nyquist frequency, can masquerade as lower frequencies; this is *aliasing*. A 70 Hz oscillation sampled at 100 Hz can appear at 30 Hz. Interpolating the resulting data to a denser grid does not undo that ambiguity. The 100 Hz resampling setting specifies the analysis grid, not proof that every source sensor supplied 100 independent readings per second.

The repository detrends $x[n]$, applies a Hann window $w[n]$, and computes the one-sided periodogram

$$P[k] = \frac{\left|\sum_n x_{\mathrm{detrended}}[n] w[n] e^{-\mathrm{i}2\pi kn/N_{\mathrm{FFT}}}\right|^2}{f_s \sum_n w[n]^2}, \tag{6.6}$$

with interior bins doubled. The stored quantity is PSD, not a complex FFT: phase and the complex Fourier coefficients are not retained. There is also a small implementation caveat. The code leaves the final real-FFT bin undoubled, which is conventional for even N_{FFT}, when that bin is Nyquist. At the default odd length $N_{\mathrm{FFT}} = 101$, the final bin is approximately 49.505 Hz rather than Nyquist, so it is half-scaled relative to the usual one-sided PSD convention. Either correct this before comparing individual bins or treat the last bin consistently as a special case.

Detrending removes a fitted straight line from the window. Otherwise slow drift can dominate low frequencies. It also removes potentially useful mean-load information, so retain means or trends as separate features if they may matter. The Hann window tapers endpoints toward

zero. Abruptly cutting a nonperiodic signal creates artificial frequency content, called spectral leakage; tapering reduces that leakage while broadening peaks.

PSD means power *per unit frequency*. For a torque channel its units are $(\mathrm{N\,m})^2/\mathrm{Hz}$. Multiplying by a bin width and summing estimates a mean-square value, or band power, over that band. This is signal power in squared sensor units, not mechanical energy or work such as $\int \tau(t)\dot{q}(t)\,dt$, measured in joules. A peak's height alone can change when the bin width or window changes; integrated band power is often the more comparable quantity.

PSD retains mean-square signal content by frequency but discards absolute phase. A single FFT assumes the process is approximately stationary over the window, obscures impact timing, and discards cross-channel phase and coherence. A useful transformation before classical ML is

$$\widetilde{P}[k] = \log(P[k] + \epsilon), \tag{6.7}$$

because raw power is usually strongly skewed.

The small $\epsilon > 0$ makes the logarithm defined at zero. Choose and report it consistently relative to channel scale. Strictly, a logarithm acts on a dimensionless ratio, so one can write $\log[(P[k] + \epsilon)/P_{\mathrm{ref}}]$ with a fixed reference power density. Changing a fixed reference only shifts the feature and is removed by centering. A log transform turns multiplicative differences into additive ones: powers of 1 and 100 are far apart linearly but only $\log 100 \approx 4.605$ apart after transformation.

Rather than relying only on dominant peaks, one can use band powers

$$B_{a:b} = \sum_{k:f_k \in [a,b]} P[k]\Delta f. \tag{6.8}$$

Broad band powers are often more stable than an exact peak whose location moves by a bin.

6.6 Raw sequence representation

A model can instead consume

$$Z = \begin{bmatrix} \mathbf{r}_1^\top \\ \vdots \\ \mathbf{r}_L^\top \end{bmatrix} \in \mathbb{R}^{L\times 15}, \qquad \mathbf{r}_t \in \mathbb{R}^{15}. \tag{6.9}$$

This retains event order, impact timing, transient shapes, cross-channel temporal relationships, and phase information lost by PSD. It requires more data and stronger regularization, but it is the natural representation for one-dimensional CNNs, temporal convolutional networks, recurrent networks, Transformers, and Mamba. A good study compares time and frequency representations instead of presuming one is superior.

For short impacts, a spectrogram or short-time Fourier transform (STFT) computes spectra in several shorter overlapping subwindows. Its axes are time and frequency, preserving when a vibration occurs. A wavelet transform uses shifted, scaled short waveforms; fine scales emphasize fast events and coarse scales emphasize slower structure. Both trade temporal precision against frequency precision. The existing single-window PSD is an efficient starting point, while a CNN on raw telemetry can learn local filters without preselecting a Fourier or wavelet basis.

Check your understanding. What information distinguishes a 10 Hz impact-induced oscillation from a constant torque offset? Which of a mean, PSD, or raw sequence preserves the location of an impact within a window?

7 Suitable first-stage models

7.1 Logistic regression

Start with a hypothesis: each standardized feature supplies some evidence for one terrain. Positive weights add evidence for rocks; negative weights add evidence for sand. The intercept b sets the baseline when all standardized features are zero. The weighted sum can be any real number, whereas a probability must lie between zero and one. The sigmoid makes that conversion.

For a feature vector x,

$$P(y = 1 \mid x) = \sigma(w^\top x + b). \tag{7.1}$$

This is a strong, low-variance baseline with interpretable coefficients. It is well suited to PCA scores and band powers, easy to regularize and calibrate, and reveals whether the problem is approximately linearly separable. If a large neural model cannot reliably beat grouped logistic regression, its extra complexity is not justified.

The name *regression* refers to modeling a numerical score even though the final task is classification. Logistic regression assumes a linear relationship in *log-odds*:

$$\log \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b.$$

Odds are probability divided by its complement. A one-unit increase in feature j, with other features fixed, multiplies the predicted odds by e^{w_j}. For standardized inputs that unit is one training standard deviation. This is an association conditional on the included features, not a claim that changing a sensor reading causes a different terrain.

Worked example: a terrain decision boundary. Suppose x_1 is standardized impact-band power and x_2 standardized mean effort, and a fitted model has

$$a = 1.5x_1 - 0.5x_2 - 0.2.$$

At $(x_1, x_2) = (1, 0)$, $a = 1.3$ and $p \approx 0.786$. At $(-1, 1)$, $a = -2.2$ and $p \approx 0.100$. The $p = 0.5$ boundary is the straight line $1.5x_1 - 0.5x_2 - 0.2 = 0$. In D dimensions it becomes a hyperplane, the higher-dimensional equivalent of a line or plane.

Linearity is a restriction on the supplied features, not necessarily on the raw measurements. Adding $x_1 x_2$ allows an interaction, such as vibration being informative only under high effort. Adding x_1^2 allows curvature. Every additional feature also creates another opportunity to fit noise. For fixed features, logistic loss with L2 regularization is convex: it has no misleading local minima. Neural networks generally lose this guarantee.

7.2 Nearest neighbours: a useful geometric reference

The k-nearest-neighbour classifier stores training feature vectors. For a new vector, it finds the k closest and uses their labels to vote:

$$\widehat{p}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i.$$

Here $\mathcal{N}_k(\mathbf{x})$ is the set of neighbour indices, not the entire dataset. With four rock examples among five neighbours, $\widehat{p} = 0.8$. Small k follows local detail and noise; large k smooths the boundary but can mix genuinely different regions.

This method makes the meaning of a representation concrete: it asks whether similar features imply similar terrain. It is sensitive to scaling and irrelevant features. In high dimensions, distances often become less informative because examples are sparse relative to the space; this is one aspect of the *curse of dimensionality*. Overlapping windows can make nearest-neighbour accuracy especially misleading under a random packet split.

7.3 Linear discriminant analysis

LDA takes the reverse route from logistic regression. Instead of directly modeling the class probability given features, it models how features are distributed within each class. Such a model is called *generative*; logistic regression is *discriminative*. Neither name implies one is intrinsically better.

LDA assumes

$$x \mid y = c \sim \mathcal{N}(\mu_c, \Sigma) \tag{7.2}$$

with a shared covariance matrix. For two classes its discriminant direction is

$$w = \Sigma^{-1}(\mu_1 - \mu_0). \tag{7.3}$$

It seeks class-mean separation relative to within-class variance. Unlike PCA, it uses labels. LDA can work well on low-dimensional PCA scores, but its covariance estimate becomes unstable when there are more spectral features than independent runs.

Let $\pi_c = P(Y = c)$ be the class prior and $p(\mathbf{x} \mid c)$ the Gaussian density of features in class c. Bayes' rule gives

$$P(c \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid c)\pi_c}{\sum_{r\in\{0,1\}} p(\mathbf{x} \mid r)\pi_r}.$$

A density is not itself a probability of one exact continuous vector; probabilities arise by integrating over regions. The ratio above produces a valid class probability.

The Gaussian log-density contains the quadratic distance $(\mathbf{x} - \mu_c)^\top \Sigma^{-1}(\mathbf{x} - \mu_c)$. With shared Σ, the $\mathbf{x}^\top\Sigma^{-1}\mathbf{x}$ terms cancel between classes. The remaining log posterior odds are

$$\mathbf{w}^\top\mathbf{x} + b, \qquad b = -\tfrac{1}{2}\mu_1^\top\Sigma^{-1}\mu_1 + \tfrac{1}{2}\mu_0^\top\Sigma^{-1}\mu_0 + \log\frac{\pi_1}{\pi_0}.$$

Thus Gaussian assumptions produce a linear boundary. The inverse covariance downweights directions that fluctuate strongly within a class and accounts for correlations between features.

For a single feature with class means 1 and 3, shared variance 1, and equal priors, the boundary is 2. If the variance increases, the probability changes more gently around the same midpoint. With many correlated FFT bins, Σ may be poorly conditioned or singular. Shrinkage replaces it by $(1 - \rho)\widehat{\Sigma} + \rho\alpha I$, blending the estimated covariance with a simpler diagonal-scale matrix. This reduces unstable inverse directions; $\rho \in [0, 1]$ controls the blend.

7.4 Support vector machines

An SVM focuses on separating classes with room to spare. Among separating hyperplanes, it prefers a large *margin*, the distance between the boundary and the nearest observations. A prediction depends on the sign of the score, so multiplying both weights and intercept by the same positive constant leaves the boundary unchanged. SVM fixes this scale by setting the margin boundaries at scores $+1$ and -1.

A linear SVM solves

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C_{\text{svm}}\sum_i \xi_i \tag{7.4}$$

subject to

$$\widetilde{y}_i(w^\top x_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \tag{7.5}$$

where $\widetilde{y}_i \in \{-1, +1\}$. It maximizes the classification margin. The labels are recoded as $\widetilde{y}_i = 2y_i - 1$ for this formula. The nonnegative ξ_i is a slack variable measuring a margin violation. The distance between the two margin planes is $2/\|\mathbf{w}\|_2$, so minimizing weight length increases that distance. The hyperparameter C_{svm} trades a wide margin against violations. Large values punish violations more strongly.

Eliminating the slack variables gives the hinge loss

$$\ell_{\text{hinge}} = \max(0, 1 - \widetilde{y}_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

Correct examples outside the margin contribute zero; near-boundary examples and errors contribute positive loss. The observations that constrain the solution are called *support vectors*.

An RBF kernel

$$\kappa(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \tag{7.6}$$

allows nonlinear boundaries and is often effective for small scientific datasets. Its output probabilities require post-hoc calibration.

RBF means radial basis function. It is a similarity measure: equal vectors have similarity 1, and dissimilar vectors approach zero. The kernel trick lets the classifier act as if examples had been mapped into a much larger feature space without explicitly creating every coordinate. Its score has the form

$$a(\mathbf{x}) = \sum_{i \in \mathcal{S}} \alpha_i \widetilde{y}_i \kappa(\mathbf{x}_i, \mathbf{x}) + b,$$

where $\mathcal{S}$ contains support vectors and the learned α_i control their influence. With large γ, influence is very local and the boundary can become wiggly. Small γ creates broader similarities. Both γ and C_{svm} require grouped validation. The raw SVM score is a margin score, not a probability.

7.5 Tree ensembles

One decision tree asks a sequence of questions, for example whether high-frequency IMU power exceeds a threshold and then whether wheel effort is large. Each terminal region, or *leaf*, assigns a class or class frequency. A tree can naturally represent rules involving several conditions and does not require a linear boundary.

Decision trees split features to reduce impurity. For binary classes, Gini impurity is

$$G = 1 - p^2 - (1 - p)^2 = 2p(1 - p). \tag{7.7}$$

Random forests average many decorrelated trees, while gradient boosting adds trees that correct previous residuals. They model nonlinear interactions, need little scaling, and form strong baselines for combined summary, spectral, and quality-control features. Because ordinary split importance is biased by correlated FFT bins and split-selection effects, held-out bandwise permutation importance is often more trustworthy.

Worked example: choosing a split. At a node with five sand and five rock examples, Gini impurity is 0.5. Suppose a proposed split creates a leaf with four sand and one rock and another with one sand and four rocks. Each has impurity $2(0.8)(0.2) = 0.32$. The weighted child impurity is also 0.32, a reduction of 0.18. Training searches features and thresholds for such reductions. This is greedy: it chooses a useful next split, not the globally best entire tree.

A deep tree can memorize very small regions. A random forest reduces this instability by fitting trees to resampled training examples and considering random feature subsets at splits. Averaging many partly different errors is more stable than trusting one tree. More trees primarily reduce averaging noise; constraints on depth and minimum leaf size still matter. With correlated packets, out-of-bag estimates from ordinary packet bootstrapping do not replace an independently grouped evaluation.

Gradient boosting combines small trees sequentially. If $F_m(\mathbf{x})$ is the current score, the next tree fits the negative loss gradient. For logistic loss that target is $y_i - \sigma(F_m(\mathbf{x}_i))$. The update is

$$F_{m+1}(\mathbf{x}) = F_m(\mathbf{x}) + \nu h_m(\mathbf{x}),$$

where h_m is the next tree and ν is its learning rate. This follows the same error-correction principle as gradient descent but adds functions instead of directly adjusting a fixed weight vector. Boosting can be very effective on tabular features, but too many stages or overly deep trees can overfit.

7.6 Multilayer perceptron

One linear score can only draw one hyperplane. A multilayer perceptron, or MLP, first creates several intermediate features and combines them. These intermediate quantities are called *hidden activations*. They are learned descriptions, not extra sensor measurements.

An MLP repeatedly applies

$$h^{(\ell+1)} = \varphi(W^{(\ell)} h^{(\ell)} + b^{(\ell)}). \tag{7.8}$$

It can model nonlinear interactions between spectral bins and channels but does not encode frequency or temporal neighbourhood unless that structure is explicitly placed in the input.

The superscript ℓ indexes layers rather than powers. $W^{(\ell)}$ is a weight matrix, $b^{(\ell)}$ a bias vector, and φ a nonlinear activation applied separately to each entry. Let $h^{(0)} = \mathbf{x}$, and let M be the number of weighted layers. Define pre-activations by

$$\mathbf{a}^{(\ell+1)} = W^{(\ell)} h^{(\ell)} + b^{(\ell)}, \qquad h^{(\ell+1)} = \varphi(\mathbf{a}^{(\ell+1)}), \quad \ell = 0, \ldots, M-1.$$

Thus $W^{(\ell)}$ connects layer ℓ to layer $\ell+1$. Here φ denotes the activation chosen for the current layer; hidden layers can use ReLU while the binary output uses a sigmoid. A common choice is ReLU, $\varphi(a) = \max(0, a)$, which passes positive values and sets negative ones to zero. Without a nonlinearity, stacking layers adds no expressive power:

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2 W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2).$$

This is still one affine transformation.

An XOR example shows why nonlinearity matters. Suppose two binary features have class 1 only when exactly one is active. No straight line separates $(0,1), (1,0)$ from $(0,0), (1,1)$. Hidden nonlinear features can describe different regions and combine them into the desired answer. In terrain data, this could represent a response pattern that appears for either high effort or high vibration but not their simultaneous occurrence. That physical interpretation is illustrative; it must be tested on data.

Training uses backpropagation. Define $\boldsymbol{\delta}^{(\ell)} = \partial \ell_i / \partial \mathbf{a}^{(\ell)}$: the loss gradient with respect to that layer's pre-activation. For a scalar output sigmoid with cross-entropy, $\delta^{(M)} = p - y$. At hidden layer $\ell = 1, \ldots, M-1$, the chain rule gives

$$\boldsymbol{\delta}^{(\ell)} = \left(W^{(\ell)\top} \boldsymbol{\delta}^{(\ell+1)}\right) \odot \varphi'(\mathbf{a}^{(\ell)}).$$

$\odot$ means elementwise multiplication. The corresponding parameter gradients, for $\ell = 0, \ldots, M-1$, are

$$\frac{\partial \ell_i}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell+1)} (h^{(\ell)})^\top, \qquad \frac{\partial \ell_i}{\partial b^{(\ell)}} = \boldsymbol{\delta}^{(\ell+1)}.$$

The weight gradient is an outer product with the preceding activations. Each weight is changed according to both the input that used it and the error it helped produce.

An MLP with D inputs and H first-layer units has $DH + H$ first-layer parameters. With 765 inputs and 64 units, that is 49,024 parameters before the classifier head. This count explains why a modest looking network can overfit a small number of terrain setups. Dropout randomly suppresses activations during training to discourage dependence on a few units. It is a regularizer, not a substitute for independent evaluation.

7.7 One-dimensional convolutional network

A convolutional neural network, or CNN, looks for short patterns using the same learned detector at many positions. For telemetry, positions are times; for RGB they are image locations. A temporal CNN might learn a wheel-effort rise followed by deceleration. The useful assumption is that this pattern has similar meaning wherever it occurs within the window.

A temporal CNN can use the following chronological cross-correlation:

$$h_k[t] = \varphi\left(b_k + \sum_{c=1}^{C} \sum_{j=0}^{K-1} W_{k,c,j} Z_{t-K+1+j,c} \right). \tag{7.9}$$

A filter can learn an effort impulse followed by wheel deceleration, a vertical acceleration burst, periodic chatter, or a characteristic hip–knee response. Weight sharing gives the inductive bias that the same local event can be recognized anywhere in the window. For the likely dataset scale, a compact 1-D CNN or temporal convolutional network is the most sensible first raw sequence neural baseline.

In this equation k selects an output filter, c a sensor channel, j a tap ordered from oldest to newest, and K the filter length. The input window runs chronologically from $t - K + 1$ to t, so this form is causal. For $t < K$, left padding is needed to define a full window. The inner sum compares that window with a learned template without reversing its taps. Deep-learning libraries generally implement this cross-correlation operation, although it is commonly called convolution. Centered versions can use future samples as well.

Worked example: a tiny filter. Consider one channel and weights $(-1, 0, 1)$ applied left to right. On readings $(1, 2, 4)$, the filter produces $-1 + 4 = 3$; on $(4, 2, 1)$, it produces $-4 + 1 = -3$. After a ReLU, the first survives and the second vanishes. This detector distinguishes a rise from a fall. Training chooses filters more useful for the labelled task rather than requiring us to specify them.

A filter spanning C channels has KC weights plus a bias. It is reused at all time locations, rather than requiring different weights for every location. This *weight sharing* is both an efficiency gain and an *inductive bias*: a built-in preference for certain kinds of solutions. It can be helpful when observations are limited.

Stacking convolutions enlarges the *receptive field*, the input interval that can influence an activation. Average pooling summarizes persistent responses; max pooling emphasizes the strongest detected event. Pooling also removes timing precision. If the label depends on when an impact occurs relative to contact, excessive pooling can remove useful information.

7.8 Temporal convolutional and recurrent models

A temporal convolutional network (TCN) uses causal, often dilated convolutions. A dilation d examines samples spaced d apart. For m layers with one stride-one convolution per layer, filter length K, and dilations d_ℓ, its receptive-field length is

$$R = 1 + (K-1)\sum_{\ell=0}^{m-1} d_\ell.$$

With $K = 3$ and dilations $1, 2, 4, 8$, $R = 31$ samples, or approximately 0.31 s at 100 Hz. At 101 samples, such a network cannot relate the whole window unless more layers or a wider kernel are used. Architectures with two convolutions per block have a correspondingly larger receptive field; count the actual layers.

A recurrent neural network (RNN) instead updates a memory:

$$\mathbf{h}_t = \tanh(W_r \mathbf{r}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}).$$

The same parameters operate at every time. The hidden state is a compressed summary of the past, and a classifier can read the last state or a pooled collection of states. Unlike a fixed-length MLP, recurrence naturally describes variable-length sequences, provided padding and state handling are correct.

Gradients through many recurrent steps contain products of derivatives and matrices. They can shrink toward zero or grow rapidly, making long-range learning difficult. LSTM and GRU models use gates to regulate memory. A schematic gated update is

$$\mathbf{h}_t = (1 - \mathbf{g}_t) \odot \mathbf{h}_{t-1} + \mathbf{g}_t \odot \widetilde{\mathbf{h}}_t.$$

The gate entries lie between zero and one; the candidate $\widetilde{\mathbf{h}}_t$ contains proposed new information. A small gate preserves old memory; a large gate replaces it. This idea will reappear in the state-space treatment of Mamba.

7.9 Attention and Transformers as a comparison

Attention lets each time step look directly at other steps. From an activation matrix $H \in \mathbb{R}^{L \times d}$, create queries $Q = HW_Q$, keys $K_{\text{att}} = HW_K$, and values $V = HW_V$. A query asks what information is needed, keys describe what each step offers, and values contain what will be retrieved. Scaled dot-product attention is

$$\text{Attention}(H) = \text{softmax}\left(\frac{QK_{\text{att}}^\top}{\sqrt{d_k}}\right) V.$$

The softmax acts along each row: $\text{softmax}(\mathbf{a})_j = e^{a_j} / \sum_r e^{a_r}$. It creates nonnegative weights summing to one. Each output is therefore a weighted mixture of value vectors. d_k is the query/key width; the square-root factor controls score scale.

An effort spike might attend to a deceleration later in the contact window. Positional information is needed because attention alone does not encode sequence order. A causal mask forbids attending to future times; an offline encoder can use both directions. A Transformer combines attention with feed-forward layers, normalization, and residual connections that add a block's input to its output.

Ordinary attention forms L^2 pairwise scores, with time cost roughly $O(L^2 d)$ for the attention operation. This can be expensive for long streams. At $L \approx 101$, it is not a pressing reason to reject attention, but its flexibility and parameter count can still be hard to justify on a small rig dataset. Architecture choice should follow a controlled comparison, not a ranking of model names.

7.10 Recommended model ladder

The recommended comparison order is:

1. majority-class and stratified-random baselines;
2. regularized logistic regression on summary and band-power features;
3. RBF SVM and gradient-boosted trees on the same features;
4. logistic regression on PCA scores;
5. a compact 1-D CNN on raw contact-centred telemetry;
6. a temporal CNN or small recurrent model;
7. Mamba as a controlled research comparison.

This order separates architectural improvements from gains caused by a better representation or by accidental leakage.

Check your understanding. Why do two linear layers without an activation collapse into one? What does a CNN assume about the usefulness of the same filter at different times? How does a recurrent state differ from an attention matrix?

8 Training and validation correctly

8.1 What a reported score should mean

Before training, complete the sentence: the model should work on a new *what*? Another packet, another lap, another day, a newly prepared terrain bed, and a different rover are increasingly different deployment conditions. A test design is a concrete answer to that question. Generalization beyond the tested conditions is a hypothesis rather than a measured result.

Training, validation, and testing have separate roles. Training fits weights. Validation chooses features, hyperparameters, thresholds, and stopping time. Testing estimates the performance of those choices once they are fixed. Repeatedly redesigning the model after seeing the test result gradually turns the test set into another validation set.

8.2 Grouped nested validation

A defensible evaluation uses an outer grouped split for final performance and an inner grouped split for model selection.

```
Outer held-out terrain setup
|-- never used for scaling, PCA, feature selection, or tuning
`-- final unbiased evaluation

Remaining setups
`-- inner grouped validation
    |-- choose model and hyperparameters
    |-- choose PCA dimension
    `-- calibrate probabilities
```

Every learned preprocessing operation must be fitted only within the current training fold: scaling, imputation, PCA, FFT-bin selection, nuisance regression, classifier fitting, and probability

calibration. Fitting PCA once on the complete dataset before cross-validation leaks information even though PCA does not use labels.

Worked example: an outer and an inner loop. Suppose there are eight independent terrain preparations, four per class, and enough recordings within each. An outer fold might reserve one preparation from each class for testing. The six remaining preparations support inner grouped splits for selecting, for example, 5, 10, or 20 PCA components and an SVM penalty. After selecting the combination, refit preprocessing and the classifier on all six training preparations, then assess the two untouched ones. Repeat across outer folds. The resulting scores estimate the entire selection procedure, not a model tuned using the outer test labels.

Stratification tries to maintain class balance; grouping keeps related examples together. Both may be desirable, but grouping must not be broken to achieve a prettier class balance. Where a physical setup contains one class, folds must contain multiple setups to include both classes.

8.3 Confounders

A classifier can learn the protocol instead of terrain. Important confounders include commanded torque, actual speed, knee stiffness and damping, terrain slope, wheel wear, battery state, travel direction, run identity, lap, absolute wheel position, camera pose, lighting, and moisture.

Absolute wheel position is particularly dangerous when sand and rocks occupy fixed angular sectors. The model can infer location rather than contact physics. Prefer phase-independent or differenced features and change terrain placement between sessions.

Initially, hold operating settings constant. Later, cross them factorially with terrain:

Terrain	Low speed	High speed	Low load	High load
Sand	yes	yes	yes	yes
Rocks	yes	yes	yes	yes

If one class is always collected at one setting, no model can statistically separate terrain from that setting.

The issue can be expressed as

$$Z = f(Y, \mathbf{u}, \mathbf{e}) + \text{measurement noise},$$

where Y is terrain, $\mathbf{u}$ operating settings, and $\mathbf{e}$ environmental conditions. If $\mathbf{u}$ always identifies Y, many different explanations fit the observed data equally well. A more advanced classifier cannot recover the missing experimental comparison.

Factorial collection varies settings across every terrain so their effects can be compared. Randomizing run order helps separate terrain from gradual changes such as battery discharge or warming motors. Repeating preparations helps separate material identity from one unusually packed bed. Metadata is not administrative overhead: it is what allows these alternative explanations to be tested.

8.4 Metrics

For terrain classification, call rocks the positive class. True positives TP are correctly predicted rock examples; false positives FP are sand predicted as rocks. False negatives FN are

rocks predicted as sand; true negatives TN are correct sand predictions. Accuracy can conceal asymmetric or imbalanced errors. Report at least

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{8.1}$$

$$F_1 = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}, \tag{8.2}$$

and balanced accuracy

$$\text{BA} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right). \tag{8.3}$$

Also report the confusion matrix, precision–recall AUC, ROC AUC, a calibration plot, performance for each held-out setup, and the Brier score

$$\text{Brier} = \frac{1}{N}\sum_i (p_i - y_i)^2. \tag{8.4}$$

Confidence intervals should bootstrap complete setups or sessions rather than individual packets.

Worked example: why accuracy is incomplete. Consider 80 sand and 20 rock test examples. A classifier that always says sand is 80% accurate but has rock recall zero and balanced accuracy 0.5. Suppose another classifier has $TP = 15, FN = 5, FP = 8, TN = 72$. Its accuracy is 87%, recall is $15/20 = 0.75$, precision is $15/23 \approx 0.652$, and balanced accuracy is $(0.75 + 0.90)/2 = 0.825$. The confusion matrix tells us what the remaining 13 errors mean.

ROC AUC measures ranking across false-positive and true-positive rates over thresholds. Precision–recall curves emphasize the positive class and depend strongly on class prevalence. AUC is not the performance at the operating threshold, and neither curve measures calibration. If a test fold contains only one class, its ROC AUC is not defined.

8.5 Calibration, uncertainty, and abstention

Two models can make identical class decisions while assigning probabilities 0.6 and 0.99. The latter will supervise RGB much more strongly under a confidence-weighted loss. A reliability diagram bins predicted probabilities and compares each bin with its observed rock frequency. The Brier score penalizes squared probability error; cross-entropy penalizes confident errors more sharply.

A simple calibration model maps a held-out score a through $\sigma(A_{\text{cal}}a + B_{\text{cal}})$. Its two parameters are fitted using validation or cross-fitted predictions. An independent calibration group is valuable because calibrating on training predictions rewards overconfidence. Calibration can change when class prevalence or the sensor/terrain distribution changes.

Uncertainty has at least two sources. *Aleatoric* uncertainty comes from ambiguity or noise, such as a mixed patch. *Epistemic* uncertainty comes from limited knowledge, such as never having trained on wet gravel. Several models trained with different seeds or group resamples can reveal some epistemic uncertainty through disagreement. They can also share the same blind spot. A probability near 0 or 1 is therefore not proof that an input resembles the training distribution.

Abstention means returning an unknown state for doubtful inputs instead of forcing a binary answer. Evaluate the accuracy–coverage trade-off: coverage is the fraction on which the model answers, and accuracy is measured on that fraction. Report both, because answering perfectly on only 2% of the data would be a weak practical result.

8.6 Learning curves and controlled ablations

A learning curve plots held-out performance against training data quantity. For this project, increase the number of independent preparations or sessions, not just the number of overlapping packets. If training performance is high and validation remains low, more varied data or stronger regularization may help. If both remain poor, inspect label quality, representation, model capacity, and whether the sensors actually contain the requested information.

An *ablation* changes one meaningful ingredient: motor-only versus IMU-only, time features versus spectra, or PCA versus no PCA. Use the same outer groups and compare paired scores. A small average improvement that reverses on half the held-out setups is weaker evidence than a consistent improvement. Bootstrap whole setups to quantify variability; thousands of resampled packets cannot compensate for only two independent test environments.

8.7 Label quality

Use structured labels instead of free-text environment descriptions. At minimum record experiment, run, a physical `terrain_setup_id`, terrain class, moisture, compaction, material batch, annotator, confidence, and boundary status. If a run crosses a material boundary, use an uncertain state or exclude a margin around the transition instead of forcing ambiguous packets into a class.

9 Building the weakly supervised visual stage

9.1 What supervision is transferred

The aim is to associate an earlier image region with a later measured interaction. A tactile teacher compresses that interaction into a target. The visual student learns which visible patterns predict the target. Consequently the quality of the student depends on three things: the teacher, the spatial/temporal correspondence, and what can actually be inferred from RGB.

For the manually labelled subset, use a direct human-label RGB baseline. This establishes whether a tactile teacher adds useful information rather than merely reproducing available labels. For additional unlabelled interactions, teacher predictions can increase the amount of visual supervision. Preserve which targets came from humans and which came from the teacher so their effects can be compared.

Let the tactile teacher output

$$q_i = q_\phi(y = 1 \mid Z_i). \tag{9.1}$$

Retaining this probability preserves the difference between targets 0.52 and 0.999 without immediately hardening them to the same class. Start with equal example weights $w_i = 1$; additional confidence weighting is optional. Let the visual student produce

$$p_i = p_\theta(y = 1 \mid I_i, M_i), \tag{9.2}$$

where M_i is the projected terrain-patch mask. A soft-label loss is

$$\mathcal{L}_{\text{vision}} = -\sum_i w_i \left[q_i \log p_i + (1 - q_i) \log(1 - p_i) \right]. \tag{9.3}$$

For one image with target q, this is the expected hard-label loss under a Bernoulli distribution with rock probability q. It also equals teacher entropy plus a divergence:

$$-q \log p - (1 - q) \log(1 - p) = H(q) + D_{\text{KL}}(\text{Bern}(q) \| \text{Bern}(p)).$$

The Kullback–Leibler divergence D_{KL} measures how much the student distribution disagrees with the teacher distribution and is minimized at $p = q$. Since $H(q)$ is fixed during student training, minimizing soft cross-entropy matches the teacher probabilities.

Worked example: retaining doubt. For an ambiguous teacher output $q = 0.6$, the student loss is approximately 0.673 at $p = 0.6$, but 1.848 at $p = 0.99$. Hardening the target to 1 would instead encourage unwarranted certainty. For a student logit a, the soft-target gradient is $d\ell/da = w(p - q)$, the same rule as supervised logistic training with the label replaced by the teacher probability.

Soft BCE already encodes the target through $p - q$: an ambiguous target pulls an overconfident student toward ambiguity. A calibrated probability near 0.5 can represent genuine aleatoric uncertainty, not an unreliable label. High entropy alone is therefore not a reason to discard that supervision.

One optional heuristic is to downweight high-entropy targets using

$$H(q_i) = -q_i \log q_i - (1 - q_i) \log(1 - q_i), \tag{9.4}$$

$$w_i = 1 - \frac{H(q_i)}{\log 2}. \tag{9.5}$$

Under this heuristic, high-entropy predictions receive little weight. The entropy is zero at $q = 0$ or 1, using the limiting convention $0 \log 0 = 0$, and is largest at $q = 0.5$. The proposed weight is therefore zero for a maximally ambiguous binary target and one for an extreme one. For $q = 0.9$, the weight is about 0.531.

This reweights the training distribution rather than repairing the probabilities. For an RGB input I associated with varying tactile targets, the weighted population target is

$$p_w^\star(I) = \frac{\mathbb{E}[wq \mid I]}{\mathbb{E}[w \mid I]}, \qquad \mathbb{E}[w \mid I] > 0,$$

instead of the unweighted target $\mathbb{E}[q \mid I]$. These need not agree. If q is uniquely determined by the student's input and $w > 0$, weighting does not change its unconstrained pointwise optimum, but it still changes which examples matter to a restricted model. Entropy weighting can therefore bias probability learning, especially across modalities.

A teacher can also be confidently wrong on unfamiliar terrain. Validate any entropy weighting against held-out teacher errors and OOD (out-of-distribution) detection, not confidence alone. Retain calibration, cross-fitting, and independent-group evaluation, and compare unweighted soft targets with both weighted targets and hard pseudo-labels.

Calibrate the tactile teacher before creating labels. Ideally, use cross-fitted predictions: every training packet receives a teacher prediction from a model that did not train on that packet's physical group. Otherwise the student inherits the teacher's in-sample overconfidence.

Cross-fitting the complete teacher. Divide the teacher-training data by physical groups. Train a teacher on all but one group, predict that omitted group, and repeat until every group has out-of-group predictions. Scaling, PCA, feature selection, calibration, and teacher hyperparameter selection must all respect the omitted group. Merely leaving the final classifier out while fitting PCA on all groups does not achieve that separation. The final visual evaluation groups remain outside both teacher and student development.

9.2 Patch classification and dense segmentation

A sensible progression is:

1. classify the saved RGB patch;
2. train a fully convolutional model using projected masks as sparse supervision;
3. add consistency regularization on unlabelled pixels;
4. attempt dense full-scene terrain maps.

The saved crops have variable dimensions. Any resize or padding policy must preserve the mask. Pixels outside the polygon are zeroed, but a genuine dark pixel may also be zero, so the explicit mask is essential. Starting with full images is risky because the network can memorize the rig, track position, background, lighting, camera geometry, or a particular material arrangement.

An RGB patch has three channels but many spatial positions. A visual CNN learns local edge, texture, and larger pattern detectors. Pretraining on a large image dataset can supply useful initial filters. First train a small classification head on these features; then, if validation supports it, fine-tune some backbone layers with a smaller learning rate. Pretraining does not guarantee that the network has learned terrain mechanics.

A *segmentation* model outputs a class distribution at each image pixel rather than one prediction for the whole crop. Let $m_{iu} = 1$ when pixel u of image i has a projected label, and zero otherwise. A global pixel-normalized soft-label objective is

$$\mathcal{L}_{\text{pixel}} = -\frac{1}{\sum_{i,u} m_{iu}} \sum_{i,u} m_{iu} w_i \left[q_i \log p_{iu} + (1 - q_i) \log(1 - p_{iu}) \right].$$

Unlabelled pixels do not become sand; they contribute no supervised term. With $w_i = 1$, this gives equal weight to each labelled pixel, so contacts with larger projected masks have more influence. To give equal weight to each contact, first average within its mask. Define $n_i = \sum_u m_{iu}$ and $\mathcal{I}_+ = \{i : n_i > 0\}$. Then use

$$\mathcal{L}_{\text{contact}} = -\frac{1}{|\mathcal{I}_+|} \sum_{i \in \mathcal{I}_+} \frac{w_i}{n_i} \sum_u m_{iu} \left[q_i \log p_{iu} + (1 - q_i) \log(1 - p_{iu}) \right].$$

Equal-contact weighting means $w_i = 1$; optional nonuniform weights change that choice. With one contact per image this also gives equal image weight. If a contact has several image views, aggregate those views within contact to avoid giving it extra weight. Empty masks are excluded before division; skip the supervised term when all masks in a batch are empty. Select $\mathcal{L}_{\text{sparse}} = \mathcal{L}_{\text{pixel}}$ or $\mathcal{L}_{\text{contact}}$ deliberately in the combined objective below.

Attaching one contact target to every projected pixel assumes that the patch is sufficiently homogeneous. A mixed patch violates that assumption. It may be better to predict a patch average, use a smaller confident mask, or mark transitions uncertain. A network cannot repair inconsistent target semantics simply by being deeper.

9.3 Mean Teacher consistency

Maintain student parameters θ_S and an exponential-moving-average teacher, with averaging coefficient $0 \le \alpha < 1$:

$$\theta_T \leftarrow \alpha \theta_T + (1 - \alpha) \theta_S. \tag{9.6}$$

For dense predictions, use the same geometric transform g for both branches, but different photometric perturbations a_1, a_2:

$$\widetilde{I}_S = a_1(g(I)), \qquad \widetilde{I}_T = a_2(g(I)).$$

Let Ω_g contain valid pixel positions in that shared transformed frame. Encourage agreement at matching positions:

$$\mathcal{L}_{\text{cons}} = \frac{1}{|\Omega_g|} \sum_{u \in \Omega_g} \left\| f_{\theta_S}(\widetilde{I}_S)_u - \text{sg}\Big[f_{\theta_T}(\widetilde{I}_T)_u \Big] \right\|_2^2 . \tag{9.7}$$

Here each output is a per-pixel class-probability vector, and sg means stop-gradient: the teacher prediction is a fixed target during backpropagation. Only the EMA update changes its parameters. Skip this term if Ω_g is empty. The complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{sparse}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}}. \tag{9.8}$$

This is useful because only a small part of each image is covered by the wheel-contact mask [4]. The functions a_1, a_2 change appearance without moving pixels, and $\lambda_{\text{cons}} \geq 0$ sets the relative weight of the consistency loss.

The consistency teacher here is a moving average of the *visual* student. It is distinct from the tactile teacher that supplied labels. A moving average reduces fluctuations in recent student updates. Consistency encourages a prediction to survive transformations that should not change terrain, such as mild brightness variation. It does not add independent ground truth and can reinforce shared errors.

Geometric augmentations must transform images, masks, and dense targets together. Comparing a flipped student's pixel with the unflipped teacher's same pixel is incorrect unless the predictions are warped into a common coordinate frame first. Photometric augmentation must remain plausible: if moisture is a meaningful target, extreme color manipulations might erase real evidence. Ramp the consistency weight only after supervised learning provides an anchor.

9.4 Cross-modal representation learning

Instead of transferring only class probabilities, learn tactile and visual embeddings

$$z_i^T = g_\phi(Z_i), \qquad z_i^V = h_\theta(I_i). \tag{9.9}$$

An InfoNCE-style objective is

$$\mathcal{L}_i = -\log \frac{\exp(\text{sim}(z_i^V, z_i^T)/\tau)}{\sum_j \exp(\text{sim}(z_i^V, z_j^T)/\tau)}. \tag{9.10}$$

This can exploit unlabelled paired packets, but negative sampling requires care: two different samples of the same material are not necessarily genuine negatives.

The embeddings $\mathbf{z}_i^T, \mathbf{z}_i^V$ are learned coordinates intended to place corresponding tactile and visual observations nearby. Here the superscripts T, V identify tactile and visual modalities; the transpose operation is always written $^\top$. The numerator rewards the true pair, while the denominator compares it with alternative pairings. Similarity is often a dot product of unit-normalized vectors; the temperature $\tau > 0$ sets how sharply the loss emphasizes the most similar alternatives. This τ is a temperature, not motor torque.

Overlapping packets can make the same physical patch appear as both a positive and an apparent negative. Group-aware sampling or multiple-positive objectives help avoid this contradiction. Contrastive learning can exploit unlabelled pairs, but the resulting coordinates still need a terrain classifier and independent evaluation to establish their usefulness.

9.5 Continuous physical targets

A binary class is a useful first milestone, but continuous targets are more valuable for planning. Candidates include slip ratio, vibration band power, motor energy per travelled distance, torque residual at controlled speed, sinkage, immobilization probability, or a learned interaction score.

For a driven wheel, when $r_{\text{eff}}\omega_{\text{wheel}} > 0$ and is safely away from zero, a longitudinal slip proxy is

$$s = 1 - \frac{v_{\text{track}}}{r_{\text{eff}}\omega_{\text{wheel}}}. \tag{9.11}$$

A stabilized signed definition is preferable near zero speed. A vision network can predict both a mean and uncertainty with

$$\mathcal{L}_{\text{NLL}} = \sum_i \left[\frac{(\mu_i - y_i)^2}{2\sigma_i^2} + \log \sigma_i \right]. \tag{9.12}$$

where $\sigma_i > 0$ is the predicted standard deviation. The first term penalizes error; the logarithmic term prevents the model from declaring infinite uncertainty everywhere. If the network instead predicts log variance $r_i = \log \sigma_i^2$, the equivalent per-example loss is

$$\ell_i = \frac{1}{2} e^{-r_i} (\mu_i - y_i)^2 + \frac{1}{2} r_i. \tag{9.13}$$

These Gaussian negative log-likelihoods omit an additive constant that does not affect training. A variance that depends on the input is called *heteroscedastic*. It primarily models conditional target variability; it does not by itself detect an unfamiliar terrain or quantify all uncertainty in the learned weights.

10 What *Where Should I Walk?* actually did

Wellhausen et al.'s 2019 paper projected robot footholds into earlier camera images, attached either manual terrain classes or automatically derived force response scores to those footholds, and trained a dense RGB predictor from sparse labels [2].

It did *not* use Mamba; the paper predates Mamba by nearly five years. For its continuous tactile target, it split six-axis foot force/torque streams into stance phases, normalized segment length, applied a Morse continuous wavelet transform, produced a 13,860-dimensional vector, separated data by four legs and two travel directions, performed PCA in each of those eight groups, and used standardized PC1 as an empirical "ground reaction score." PC1 explained 37.5% of variance on average, but the paper did not claim that it represented one specific physical parameter.

For vision, the paper used a modified ERFNet encoder–decoder CNN [3], skip connections, cross-entropy for terrain classes, heteroscedastic Gaussian NLL for the continuous score, and Mean Teacher consistency for sparsely labelled images.

Where Should I Walk?	LegWheel project analogue
Foot force/torque stance phase	Motor/joint and IMU contact window
Wavelet features	Existing FFT or raw temporal features
Projected foot contact	Projected annular wheel patch
PCA ground-reaction score	Sand/rock classifier or continuous response score
ERFNet predictor	Patch classifier or segmentation student
Sparse foothold regions	Saved RGB patch masks

11 Mamba-derived models

Mamba was introduced by Gu and Dao in 2023 as a selective state-space sequence model [5]. It begins with classical dynamical systems.

11.1 Start with a memory, not an architecture name

Suppose a torque signal briefly jumps when the wheel strikes a rock. A useful model should remember the jump long enough to relate it to the following deceleration, but need not preserve every earlier reading forever. A hidden state is a compact collection of numbers that carries such information forward.

The simplest scalar memory is an exponential average:

$$h_t = \rho h_{t-1} + (1-\rho)x_t, \qquad 0 < \rho < 1.$$

The current input contributes $1-\rho$, and an input j steps old contributes $(1-\rho)\rho^j$. A larger ρ remembers longer but reacts more slowly. With $\rho = 0.8$, $h_0 = 0$, and inputs $0, 5, 0, 0$, the states are $0, 1, 0.8, 0.64$. The impulse leaves a fading trace.

One memory cannot represent every timescale. A vector of memories can combine fast components that react to impacts and slow components that track load. State-space models generalize this idea. Their learned states need not correspond to named physical properties, even when the equations resemble a physical dynamical system.

11.2 Continuous state-space system

For this derivation, consider one scalar input $x(t)$, an N_s-entry state $h(t)$, and a scalar output $o(t)$:

$$\dot{h}(t) = Ah(t) + Bx(t), \tag{11.1}$$

$$o(t) = Ch(t) + Dx(t). \tag{11.2}$$

Here $x(t)$ is observed telemetry, $h(t)$ is latent memory, A governs memory evolution, B determines how input enters memory, C maps memory to output, and D is a direct input-to-output path. This resembles a learned physical dynamical system.

The dimensions are $A \in \mathbb{R}^{N_s \times N_s}$, $B \in \mathbb{R}^{N_s \times 1}$, $C \in \mathbb{R}^{1 \times N_s}$, and scalar D. Here C is an output matrix, not the earlier channel count; D is a direct-path coefficient, not the earlier feature count. These are standard local state-space names. The output $o(t)$ is an internal learned feature, not yet a sand/rock label. A final classifier maps a collection of such features to a class probability.

The derivative $\dot{h}(t)$ describes how quickly memory changes. For a scalar $A < 0$, old state decays rather than growing without bound. Several negative decay rates produce several memory timescales. Unconstrained unstable dynamics can amplify small disturbances, which is why practical parameterizations restrict or structure these dynamics.

11.3 Discretization

Sensors produce discrete observations. To connect the continuous equation to samples, hold the input constant over an interval of length Δ; this is the zero-order-hold assumption. Solving the differential equation over that interval gives

$$h(t+\Delta) = e^{\Delta A}h(t) + \left(\int_0^{\Delta} e^{A\tau}B\,d\tau\right)x_t.$$

The matrix exponential is the generalization of e^a: $e^{\Delta A} = I + \Delta A + (\Delta A)^2/2! + \cdots$. It describes the cumulative evolution during the interval, not entrywise exponentiation of an arbitrary matrix.

Assuming that input is constant for a step of duration Δ,

$$\bar{A} = e^{\Delta A}, \tag{11.3}$$

$$\bar{B} = \int_0^{\Delta} e^{A\tau} B \, d\tau. \tag{11.4}$$

If A is invertible,

$$\bar{B} = A^{-1}(e^{\Delta A} - I)B. \tag{11.5}$$

The recurrence becomes

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \tag{11.6}$$

$$o_t = Ch_t + Dx_t. \tag{11.7}$$

Worked example: a physical time constant. Let $A = -1/\tau_m$ and $B = 1/\tau_m$ for one state, where $\tau_m > 0$ is a memory time constant. Then

$$h_t = e^{-\Delta/\tau_m} h_{t-1} + (1 - e^{-\Delta/\tau_m})x_t.$$

For a 0.01 s sampling interval and $\tau_m = 0.10$ s, the retention factor is $e^{-0.1} \approx 0.905$. After 0.10 s, an unrefreshed trace is reduced to $e^{-1} \approx 0.368$ of its initial value. This connects an algebraic coefficient to an intuitive duration. When $A = 0$, the integral definition still works and gives $\bar{B} = \Delta B$; the inverse formula is simply unavailable.

11.4 Connection to convolution

Unrolling gives

$$h_t = \bar{A}^t h_0 + \sum_{k=1}^{t} \bar{A}^{t-k} \bar{B} x_k. \tag{11.8}$$

Ignoring the initial-state term,

$$o_t = \sum_{k=1}^{t} C\bar{A}^{t-k} \bar{B} x_k + Dx_t. \tag{11.9}$$

With lag kernel

$$K_0 = C\bar{B} + D, \qquad K_j = C\bar{A}^j \bar{B} \quad (j \geq 1), \tag{11.10}$$

the system can be evaluated as a recurrence for streaming inference or as a convolution $o = x * K$ for parallel training.

For example, with $\bar{A} = 0.8, \bar{B} = 0.2, C = 1, D = 0$, the kernel is $(0.2, 0.16, 0.128, \ldots)$. An input of 5 at one instant produces outputs $1, 0.8, 0.64, \ldots$. We have computed exactly the same operation in two ways: update a state step by step, or weight the complete input history with a fixed decay-shaped filter.

This equivalence relies on fixed parameters and linearity. It is called a linear time-invariant system: shifting an input shifts its response, and adding two inputs adds their responses. It supplies a principled sequence filter, but the same memory policy applies during steady rolling, an impact, and a sensor disturbance. A useful next step is to let the observed content influence the memory policy.

11.5 Mamba selectivity

A fixed state-space system uses the same dynamics everywhere. Mamba makes key parameters depend on the current input:

$$B_t = s_B(x_t), \qquad C_t = s_C(x_t), \tag{11.11}$$

$$\Delta_t = \mathrm{softplus}(\Delta_0 + s_\Delta(x_t)). \tag{11.12}$$

Then

$$\bar{A}_t = e^{\Delta_t A}, \tag{11.13}$$

$$\bar{B}_t = \int_0^{\Delta_t} e^{A\tau} B_t \, d\tau, \tag{11.14}$$

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \tag{11.15}$$

$$o_t = C_t h_t + D x_t. \tag{11.16}$$

This gives content-dependent memory: preserve a steady rolling baseline, admit an impact transient, forget obsolete state after a regime change, or emphasize different latent components during slip and vibration. A scalar intuition is the learned gate

$$h_t = (1 - g_t) h_{t-1} + g_t x_t, \qquad g_t = \sigma(W x_t + b), \tag{11.17}$$

although the full Mamba model is richer than this scalar example.

These equations are a simplified, single-channel explanation. Practical Mamba blocks use structured, commonly diagonal, state dynamics rather than an arbitrary dense A. Moreover, the learned Δ_t is an internal timescale or gate; it is not automatically the physical time elapsed between sensor samples. Standard Mamba does not inherently understand irregular timestamps or missing observations. Those require regular resampling, masks, elapsed-time features, or a model designed explicitly for irregular sampling.

The conceptual roles differ. B_t controls what is written into the state, C_t controls what is read out, and Δ_t changes how strongly the state advances or forgets. With negative diagonal A, larger Δ_t makes $e^{\Delta_t A}$ smaller, so old state is retained less. The learned projections s_B, s_C, s_Δ turn input features into these controls. Softplus, $\log(1 + e^a)$, ensures a positive step parameter.

The exact input coefficient depends on discretization. The integral above explains zero-order hold; practical selective-SSM implementations may use a simpler input injection proportional to $\Delta_t B_t x_t$. Understanding this distinction prevents treating the pedagogical equations as a line-by-line specification of every implementation.

Worked example: selective memory. In the scalar gated model, take $h_{t-1} = 1$. A new input $x_t = 5$ with gate $g_t = 0.8$ changes memory to $0.2(1) + 0.8(5) = 4.2$. If the next input is zero and its gate is 0.05, memory becomes $0.95(4.2) = 3.99$: the event is preserved despite a quiet sample. If the gate were always 0.8, it would fall immediately to 0.84. A learned gate could instead reject an isolated nuisance input by keeping its write strength small. These are possible behaviors, not guaranteed interpretations of a trained model; inspect them empirically.

11.6 Selective scan

Because parameters vary with time, Mamba cannot use one fixed convolution kernel. Nevertheless, affine state updates compose associatively. If

$$h' = A_1 h + b_1, \qquad h'' = A_2 h' + b_2, \tag{11.18}$$

then

$$(A_2, b_2) \circ (A_1, b_1) = (A_2 A_1,\ A_2 b_1 + b_2). \tag{11.19}$$

This permits a parallel prefix scan while retaining linear scaling with sequence length. Roughly,

$$\text{cost}_{\text{Mamba}} = O(LdN_s), \tag{11.20}$$

for model width d, sequence length L, and state size N_s, whereas ordinary self-attention has a quadratic dependence on L.

Although recurrence looks sequential, a sequence of affine transformations can be regrouped without changing its result. For scalars, take $f_1(h) = 0.5h + 1$ and $f_2(h) = 0.2h + 3$. Their composition is $f_2(f_1(h)) = 0.1h + 3.2$. A third transform can be combined with either that result or the second transform first. Associativity lets a parallel prefix algorithm combine blocks in a tree rather than process every step serially.

In structured diagonal states, composition uses elementwise products rather than expensive dense matrix multiplication. The scan does not remove temporal dependence; it computes the same dependence with a more efficient schedule. The rough linear expression above refers to the state update at fixed width and state dimension. Input/output projections also cost roughly $O(Ld^2)$; practical speed depends on hardware, batch size, kernel implementation, and overhead.

11.7 Mamba block

A typical block contains an input projection; a main path and gate; a short causal convolution; SiLU activation; a selective SSM; multiplicative gating; an output projection; normalization; and a residual connection. Important hyperparameters are model width, state dimension, local convolution width, inner expansion, and number of blocks.

The short convolution detects local changes; the SSM carries information over longer context; the gate modulates the resulting features. SiLU is $\text{SiLU}(a) = a\sigma(a)$, a smooth nonlinear activation. Normalization controls activation scales, and the residual path lets a block learn a correction to its input. Parameters are learned by backpropagation through all these operations using the classification loss. The state-space structure supplies a useful bias, not an explicit terrain physics simulator.

11.8 The likely Mamba terrain reference

The likely reference is LaRocque et al.'s 2024 IROS paper *Proprioception Is All You Need: Terrain Classification for Boreal Forests* [7]. It used raw IMU sequences at 100 Hz and wheel velocity plus motor current at 6.5 Hz, with 1.7-second samples. One branch processed IMU and another wheel signals. Each branch used a linear projection followed by one Mamba block; the final causal states were concatenated and classified. Its CNN baseline consumed STFT spectrograms.

The CNN was better on each individual dataset, while Mamba became more competitive as training quantity and diversity increased. The appropriate lesson is that Mamba is a model to benchmark, not an automatic upgrade. The study's validation used five-second temporal partitions rather than fully independent physical terrain setups, so its results are evidence for that study, not a guaranteed outcome for this rig.

11.9 Mamba-2 and the matrix view of memory

There is another way to describe a recurrent memory: how much does each earlier input contribute to each later output? For a time-varying SSM with zero initial state, expansion gives

$$o_t = \sum_{j=1}^{t} C_t \left(\prod_{r=j+1}^{t} \bar{A}_r \right) \bar{B}_j x_j + D x_t,$$

where the product is ordered as $\bar{A}_t \bar{A}_{t-1} \cdots \bar{A}_{j+1}$, and an empty product is I. This defines a lower-triangular sequence-mixing matrix M: row t tells us which past samples influence output t. Entries above the diagonal are zero because a causal system cannot use future samples.

For a scalar recurrence $h_t = a_t h_{t-1} + b_t x_t$ and $o_t = c_t h_t$, three steps produce

$$\begin{bmatrix} o_1 \\ o_2 \\ o_3 \end{bmatrix} = \begin{bmatrix} c_1 b_1 & 0 & 0 \\ c_2 a_2 b_1 & c_2 b_2 & 0 \\ c_3 a_3 a_2 b_1 & c_3 a_3 b_2 & c_3 b_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

The entries cannot vary independently: they share the same intervening memory factors. Distant blocks have low-rank structure because information from the past must pass through a limited-size state. Matrices with this kind of structure are called *semiseparable*.

Mamba-2 uses state-space duality (SSD) to exploit related recurrent and matrix forms efficiently [6]. In its core structure, a scalar decay is shared across state coordinates within a head, while different heads can carry different dynamics. The resulting restrictions make chunked matrix computations efficient: compute interactions within a chunk, summarize the chunk into state, and pass that state onward.

Attention also forms a matrix mixing sequence positions, but ordinary softmax attention and this structured recurrent matrix are not identical. They impose different constraints on how information flows. The useful intuition is that a recurrence and a large sequence transformation can be two views of the same restricted computation. Mamba-2 is therefore an architectural and computational development, not a special terrain-labeling method or a promise of improved accuracy on short windows.

11.10 Application to LegWheel

Reasonable comparisons include one 15-channel Mamba sequence, separate motor/joint and IMU branches, causal final-state classification, and a bidirectional offline encoder. The last distinction matters: existing windows are centred on contact and contain samples after first contact. An offline model can exploit both halves, but a real-time rover must wait for the window or be trained on causal trailing data.

At approximately 101 samples, Mamba's linear-complexity advantage is not important. Its value would have to come from better temporal representation. It also does not automatically become invariant to speed, load, torque setpoint, or sensor placement.

Mamba-2's faster core becomes more relevant for much longer sequences or larger models. Ordinary one-dimensional Mamba should not be assumed to understand images. Vision adaptations include bidirectional Vision Mamba/Vim [8] and VMamba's four two-dimensional scan routes [9]. A pretrained CNN remains a safer first RGB student for a modest custom dataset.

Check your understanding. In $h_t = 0.9 h_{t-1} + 0.1 x_t$, what fraction of an unrefreshed state remains after ten steps? Why does input-dependent memory prevent reuse of one fixed convolution kernel? Which part of a Mamba pipeline turns its output features into a terrain class?

12 Principal component analysis

Imagine two neighbouring frequency bands that rise and fall together. Keeping both gives the classifier two partly redundant descriptions. PCA rotates the coordinate system so one new coordinate captures their common increase and another captures their difference. If the difference is mostly noise, we may keep the first coordinate and discard the second. The new coordinates are combinations of the original features, not selected original bins.

PCA stands for principal component analysis. A *principal direction* or *loading vector* specifies a direction in feature space. A *score* is one example's coordinate along that direction. A *component* is sometimes used for either one, so always specify which object is being plotted. A loading vector has one entry per input feature; a vector of scores has one entry per example.

PCA finds directions of greatest input variance without using class labels. Let $X \in \mathbb{R}^{N \times D}$ contain one flattened spectral vector per packet and centre each feature:

$$X_c = X - \mathbf{1}\mu^\top. \tag{12.1}$$

Here $\mu \in \mathbb{R}^D$ is the vector of training feature means, and $\mathbf{1} \in \mathbb{R}^N$ is a column of ones. Their outer product repeats the same mean row N times. Centering makes the origin the average observation; without it the first direction could point toward a large baseline rather than describe variation around that baseline. The covariance matrix is

$$S = \frac{1}{N-1} X_c^\top X_c. \tag{12.2}$$

Its diagonal entries are individual feature variances, and off-diagonal entries are covariances. For a unit-length direction v, the score vector is $X_c v$. Since the features are centered, score variance is

$$\frac{1}{N-1}(X_c v)^\top (X_c v) = v^\top S v.$$

This derives the quantity PCA maximizes. We require unit length because otherwise multiplying v by a large constant would increase variance without finding a more meaningful direction. The first principal direction solves

$$v_1 = \arg\max_{\|v\|=1} v^\top S v. \tag{12.3}$$

With Lagrangian

$$\mathcal{J}(v, \lambda) = v^\top S v - \lambda(v^\top v - 1), \tag{12.4}$$

differentiation gives

$$Sv = \lambda v. \tag{12.5}$$

More explicitly, the derivative is $2Sv - 2\lambda v = 0$. The extra variable λ is a Lagrange multiplier enforcing $v^\top v = 1$. The resulting equation says the covariance transformation does not turn v; it only stretches it by λ. This is the definition of an eigenvector and its eigenvalue. Choose the eigenvector with largest eigenvalue for PC1.

PC2 maximizes remaining variance subject to being perpendicular to PC1; later directions repeat this condition. Orthogonality means $v_j^\top v_k = 0$ for different directions. These scores are uncorrelated on the training data. They need not be statistically independent, and orthogonality is a geometric constraint rather than a guarantee of different physical causes. Thus PCA directions are covariance eigenvectors, and their variances are the eigenvalues. The projected coordinates are

$$Z_{\mathrm{PCA}} = X_c V_q, \tag{12.6}$$

where $V_q = [v_1, \ldots, v_q]$. Explained-variance ratio is

$$r_j = \frac{\lambda_j}{\sum_k \lambda_k}. \tag{12.7}$$

12.1 Worked two-band example

Suppose two centered log-power features have estimated covariance

$$S = \begin{bmatrix} 4 & 3 \\ 3 & 4 \end{bmatrix}.$$

Their variances are both 4 and covariance is 3, so correlation is $3/4$. The unit vectors

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

are eigenvectors with eigenvalues 7 and 1. Verify this by multiplying Sv_1 and Sv_2. The first component represents common power in both bands; the second represents contrast between bands. The explained-variance ratio of PC1 is $7/(7+1) = 87.5\%$.

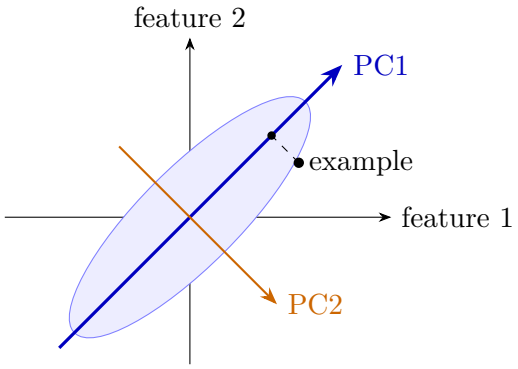


Figure 1: Schematic PCA geometry. The long direction captures common variation. The dashed segment is the error when an example is projected onto PC1 only. The ellipse illustrates a covariance cloud; it is not a plot of rig data.

For an example $\mathbf{x}_c = (2, 1)^\top$, the scores are

$$z_1 = v_1^\top \mathbf{x}_c = \frac{3}{\sqrt{2}} \approx 2.121, \qquad z_2 = v_2^\top \mathbf{x}_c = \frac{1}{\sqrt{2}} \approx 0.707.$$

Keeping only PC1 gives reconstruction $\hat{\mathbf{x}}_c = z_1 v_1 = (1.5, 1.5)^\top$. The residual is $(0.5, -0.5)^\top$, with squared length 0.5. We removed the band difference while preserving their common level. Whether that is good for terrain classification depends on which of those properties separates the classes.

12.2 SVD and reconstruction views

If

$$X_c = U \Sigma V^\top, \tag{12.8}$$

then PCA loadings are the columns of V and

$$\lambda_j = \frac{\sigma_j^2}{N-1}. \tag{12.9}$$

SVD avoids explicitly forming a potentially large covariance matrix. PCA also minimizes rank-q reconstruction error:

$$\min_{\operatorname{rank}(\widehat{X}) \le q} \|X_c - \widehat{X}\|_F^2. \tag{12.10}$$

This explains its value for compression and denoising.

The singular value decomposition (SVD) writes the same centered matrix using three coordinated transformations. The columns of V give feature-space directions, the diagonal entries of Σ give their sizes, and U describes how examples use those directions. The score matrix is $X_cV = U\Sigma$. Squaring a singular value and dividing by $N-1$ gives its score variance, which is the corresponding covariance eigenvalue.

With q retained components, reconstruction is

$$\widehat{X}_c = (X_cV_q)V_q^\top.$$

Among all q-dimensional linear subspaces, these directions minimize total squared reconstruction error. The discarded variance is the sum of the omitted eigenvalues. This is why maximizing retained variance and minimizing reconstruction error are two views of the same problem.

The centered matrix has rank at most $\min(N-1, D)$, so there cannot be more nonzero principal components than that bound. If there are many more bins than independent observations, estimated directions can be unstable even though an SVD can still be calculated. Numerical computability and scientific reliability are different properties.

12.3 PCA is not feature importance

PCA asks which combinations of bins vary most. It does not ask which bins best separate sand from rocks. PC1 may reflect wheel speed, global vibration, commanded torque, sensor gain, recording day, or terrain. A low-variance direction can still be highly discriminative. The ground-reaction result in *Where Should I Walk?* was useful only because the authors empirically checked its relation to terrain and navigation outcomes.

Return to the two-band example. Imagine sand and rocks have similar total power but rocks consistently shift energy from band 2 into band 1. The class signal then lies along PC2, the difference direction, even though PC2 explains only 12.5% of variance. A rule retaining only enough components to explain 85% would throw away that useful distinction.

An explained-variance curve answers a compression question. Grouped classification validation answers a prediction question. Choose the PCA dimension using the latter when classification is the objective. A two-dimensional score plot is exploratory evidence, not a substitute for that test. Color points by class, but also by run, speed, and session to see whether a visually striking separation has another explanation.

12.4 Scaling choices

Standardized PCA uses

$$X'_{ij} = \frac{X_{ij} - \mu_j}{s_j}. \tag{12.11}$$

For FFT data, raw PSD PCA emphasizes high-energy channels and bands; log-PSD reduces dynamic range; standardizing every bin gives weak noisy bins equal initial variance; and block scaling can prevent motor or IMU channels from dominating through units or dimensionality. Compare centred log-PSD, channel-standardized log-PSD, and separately block-scaled motor and IMU log-PSD. This choice is part of the scientific question. Thousands of overlapping packets from one long run can otherwise dominate both PCA and the classifier even when the final split is grouped. Packet thinning or group weights can give independently prepared terrain setups comparable influence.

Whitening goes one step further than PCA: divide score z_j by $\sqrt{\lambda_j}$ so each retained component has unit training variance. This can help some distance-based models, but it also amplifies

directions with small eigenvalues, including noise. Standard PCA does not require whitening. Treat it as a distinct, validated preprocessing decision.

There is no uniquely correct scaling for mixed motor and IMU spectra. Equalizing every bin gives each coordinate equal initial variance; equalizing each channel gives each physical signal a comparable total contribution; preserving physical scale asks which channels dominate actual response. State the question first, then choose a scaling that corresponds to it.

A conceptual PCA procedure. Using only the current training fold, determine which bins/channels are valid, handle missing values, apply the chosen log transform, estimate centering/scaling, calculate loading vectors, and retain the selected dimension. Save those training quantities. Apply exactly that transformation to validation and test examples; do not recenter each test run separately unless test-time adaptation was explicitly part of the protocol.

Check your understanding. For $S = \left[\begin{smallmatrix} 4 & 3 \\ 3 & 4 \end{smallmatrix}\right]$, why is the sum direction more variable than the difference direction? Can changing a component's sign change its explanatory power? Why might preserving 99% of variance still preserve irrelevant variation?

13 PCA at different FFT resolutions

The code selects

$$L = \text{round}(T_{\text{win}} f_s) + 1 \tag{13.1}$$

because both endpoints of the time window are included. At $f_s = 100$ Hz:

Window	Samples L	Spacing f_s/L	Highest bin
0.25 s	26	3.846 Hz	50.000 Hz
0.50 s	51	1.961 Hz	49.020 Hz
1.00 s	101	0.990 Hz	49.505 Hz
2.00 s	201	0.498 Hz	49.751 Hz

The theoretical Nyquist bound remains 50 Hz. For odd L, the highest DFT bin is slightly below it. The bin-spacing trade-off is

$$\Delta f \approx \frac{1}{T_{\text{win}}}. \tag{13.2}$$

Here "spacing" is precise: the actual ability to distinguish nearby sinusoids also depends on the analysis window's main-lobe width. A Hann window broadens that main lobe, so resolving power is coarser than one bin. Zero-padding interpolates a spectrum but creates no new resolving power.

Long windows give finer bin spacing but smear transients, include more terrain outside the target patch, increase boundary rejection, and create more overlap. Short windows localize contact better but merge nearby frequencies. Increasing sample rate raises the Nyquist limit; it does not by itself improve resolution unless duration or sample count also increases. Changing window duration therefore changes both spectral spacing and the physical time/terrain context. To isolate spectral granularity, hold duration fixed and aggregate the same periodogram onto differently sized common frequency bands; use varying durations as a separate context-plus-resolution experiment.

13.1 Rigorous multi-resolution experiment

For a real signal of length L, the stored nonnegative frequency count is $B = \lfloor L/2 \rfloor + 1$, where $\lfloor \cdot \rfloor$ rounds down. The flattened feature count is $D = BC$. Thus the one-second default has $B = 51$ and $D = 765$, whereas the two-second version has $B = 101$ and $D = 1{,}515$. A longer window changes input dimension as well as context.

For each resolution r:

1. construct
$$X^{(r)} \in \mathbb{R}^{N_{\text{packets}} \times N_{\text{bins}}^{(r)} C}; \tag{13.3}$$
2. match observations across resolutions by experiment, run, and sensor timestamps rather than packet ID, because configuration changes alter hashes;
3. keep all resolutions of a raw run in the same outer split and compare them on the intersection of packets valid at every resolution;
4. report the retention rate for each resolution separately, because longer windows reject more boundary packets;
5. transform valid powers with $\log(P + \epsilon)$;
6. fit missing-value handling, scaling, and PCA only on training groups;
7. select the number of PCs in inner grouped validation rather than by an automatic 95% variance threshold;
8. fit a simple classifier on scores;
9. compare held-out performance using identical outer groups;
10. inspect loadings only after reproducible classification is established;
11. repeat across independently prepared terrain setups.

The pipeline also requires $T_{\text{spectral}} \geq T_{\text{local}}$, so a short candidate window can be invalid for a slow run. Rejected channels appear as NaNs in spectral arrays; exclusion or imputation must remain inside each training fold.

13.2 Interpreting loadings

If v_j is a component loading, $v_{j,c,k}$ gives the contribution of channel c and frequency bin k. Channel-band loading energy is

$$L_{j,c,[a,b]} = \sum_{k : f_k \in [a,b]} v_{j,c,k}^2. \tag{13.4}$$

This can reveal that a component represents global IMU vibration, contrasts wheel effort with vertical acceleration, or emphasizes a narrow periodic component. Component sign is arbitrary: v_j and $-v_j$ are equally valid.

Suppose a linear classifier on PCA scores is

$$s = a^\top z + b, \qquad z = V_q^\top x'. \tag{13.5}$$

Then

$$s = (V_q a)^\top x' + b, \tag{13.6}$$

so

$$w_{\text{spectral}} = V_q a \tag{13.7}$$

maps the classifier back into standardized FFT space. If $x' = \text{diag}(1/\boldsymbol{\sigma})(x - \mu)$, weights and intercept in original units are

$$w_{\text{original}} = \text{diag}(1/\boldsymbol{\sigma}) V_q a, \tag{13.8}$$

$$b_{\text{original}} = b - w_{\text{original}}^{\top} \mu. \tag{13.9}$$

Here $\boldsymbol{\sigma}$ contains the saved feature scaling factors, and $1/\boldsymbol{\sigma}$ means their elementwise reciprocals. This is more directly relevant to classification than PCA loadings alone, but it remains associative rather than causal and may be unstable across correlated neighbouring bins.

13.3 Better tests of what matters

Combine PCA with grouped permutation importance, frequency-band ablation, leave-one-channel-out experiments, sparse logistic regression, partial least squares, supervised LDA, mutual information, and stability selection across grouped bootstraps. For example, remove all 15–30 Hz bins and retrain using the same outer folds. A change in held-out performance is more meaningful than a large loading. Joint band ablation or bandwise permutation is usually more reliable than interpreting one coefficient. A label-permutation test must shuffle at the physical-setup level and respect any experimental strata needed for exchangeability; packetwise shuffling gives a misleading null distribution for correlated windows.

Permutation importance means measuring how a fixed model's performance changes when a feature is shuffled across held-out examples, breaking its association with the label. Band ablation means removing the band and retraining, allowing other features to compensate. These answer different questions. With strongly correlated bands, shuffling one band can either have little effect because another substitutes for it or create implausible combinations. Shuffle related channel-band features together and report the grouping.

When comparing resolutions, compare physically defined bands such as 5–15 Hz, not the same bin indices. Integrate PSD across the band; near a band edge, account for how bins overlap the requested interval rather than simply averaging an unequal number of bins. Refit each representation's PCA within the same training groups. Components can change order or rotate within nearly equal-variance subspaces, so identical component numbers need not mean identical physical patterns across resolutions.

A loading plot should identify the channel, frequency axis, scaling convention, explained variance, and variability across independent fits. A large loading that changes drastically with the held-out session is weak evidence for a stable terrain signature. Classifier back-projection plus joint band ablation gives a more direct answer to which spectral properties matter for this particular prediction task.

13.4 Physical frequency hypotheses

If wheel angular rate is ω,

$$f_{\text{rotation}} = \frac{|\omega|}{2\pi}. \tag{13.10}$$

For N_g equally spaced grousers or repeated contact elements,

$$f_{\text{pass}} = N_g f_{\text{rotation}}, \tag{13.11}$$

with harmonics at integer multiples. A band that moves with wheel speed is probably linked to wheel mechanics rather than an invariant material signature. Order-normalized frequency

$$o = \frac{f}{f_{\text{rotation}}} \tag{13.12}$$

can separate wheel-synchronous structure from broadband terrain response. Use it only when $f_{\text{rotation}} \geq f_{\min} > 0$, with a minimum threshold above the measured speed-noise floor. A threshold of 0.1 Hz is an illustrative study choice, not a validated rig default. At zero or below the chosen threshold, mark order features invalid rather than divide by a near-zero rate.

14 Code-specific data integrity issues

14.1 Projection constants disagree

Production kinematics currently uses $h_b = 0.0661$ m and camera offset from beam centre 0.054 m in `src/legwheel_kinematics/legwheel_kinematics/config.py`. The README and tests instead state $h_b = 0.0761$ m and offset 0.04 m. These values directly alter RGB/depth patch projection. Before projected patches become visual labels, determine which measurements match the physical rig.

14.2 Cross-stream clocks need empirical validation

Camera and IMU messages use vendor header timestamps; motor messages use the ROS node clock; headerless encoder and requested-torque topics fall back to bag receipt time. Processing assumes sufficiently aligned clocks. RGB/depth pairing can be correct while camera-to-contact alignment remains biased, so offset and drift should be measured on real recordings.

14.3 Motor read failures and clipping

A failed motor read can cause the previous valid value to be republished with a fresh timestamp. Downstream validity masks may not recognize this as stale. Audit unexpected flat sequences and duplicated values. Hip and knee velocity feedback is also clipped to configured software limits; a classifier may learn clipping frequency rather than terrain response.

14.4 Selection bias

A packet is retained only if the complete RGB and depth patch is visible. Stage 2 additionally requires adequate depth and a valid RANSAC plane. If rough rocks produce more depth failures than sand, retention becomes class-dependent. For a tactile-only study, derive spectra from `telemetry.npz` independently of plane-fitting success if practical. At minimum, report Stage 2 rejection rate and reason separately for every class.

14.5 Structured labels do not yet exist

The repository has free-text environment descriptions and run notes but no sandy/rocky field in packet metadata, manifests, or feature records. Define an immutable label table before a large campaign so class semantics remain reproducible.

15 Recommended research sequence

Phase 1: demonstrate tactile separability

Collect independently prepared sand and rock setups, hold control settings fixed, label homogeneous runs, exclude transitions and low-quality packets, establish grouped logistic/SVM/tree baselines, test a compact 1-D CNN, and measure calibration on unseen physical setups. The success criterion is group-held-out performance rather than random packet accuracy.

Phase 2: understand the signal

Produce learning curves by number of independent setups; compare motor-only, IMU-only, and fused inputs; compare time, FFT, and combined representations; run multi-resolution PCA and channel/band ablations; check dependence on speed, load, and wheel position; and add continuous slip, vibration, and energy targets.

Phase 3: build the tactile teacher

Choose the best calibrated model, generate cross-fitted soft predictions, validate abstention against teacher errors and out-of-distribution cases, and preserve both probability and uncertainty. High entropy alone can represent genuine ambiguity and need not trigger abstention.

Phase 4: train the RGB student

Begin with the saved masked RGB patch, use soft tactile probabilities as targets, hold out complete material setups and sessions, compare a pretrained CNN with a small from-scratch model, add Mean Teacher consistency, and evaluate against human labels rather than only agreement with the tactile teacher.

Phase 5: dense visual terrain learning

Train a fully convolutional student on sparse projected masks; optionally use depth geometry as an auxiliary target or training-time privileged signal; validate on dense manual annotations from unseen scenes; convert predictions into a calibrated terrain cost map; and add unknown-terrain detection and safe abstention.

Phase 6: advanced sequence models

Only after baselines and data integrity are trustworthy, compare TCN, recurrent, Transformer, Mamba, and Mamba-2 encoders; test causal versus centred windows; compare fused and separate motor/IMU branches; control parameter count and training budget; and report performance versus independent runs rather than packets.

16 A complete worked study design

The following design joins the algorithms into one experiment. Its numbers are an illustrative planning example, not a claim that this amount of data will be sufficient.

Prepare twelve independent terrain setups, six per class, with material rearrangement between preparations and recordings across multiple days. Use the same initial control settings and record those settings explicitly. Assign each homogeneous run a manual class, retaining transition regions as uncertain. Keep all runs and packets from one preparation in the same group.

Reserve two preparations per class as a final test set. The remaining four per class support grouped training and validation. Before inspecting test performance, define the candidate representations and model-selection rule. For a compact example, use two log-band powers from each of 15 channels and two time summaries from each channel. This gives $D = 2(15) + 2(15) = 60$ features. Choose bands for a stated physical reason and validate the choice without the test set.

Compare standardized logistic regression with an RBF SVM on the same inputs. As a separate spectral experiment, flatten the full 51×15 PSD, fit training-only PCA, and validate several component counts. Compare a compact 1-D CNN on the 101×15 raw window to ask whether temporal order adds information. A Mamba comparison answers a later, more specific question about learned memory.

Suppose inner validation selects five PCA scores and a regularized logistic classifier. One unseen packet then follows this chain:

$$\underbrace{Z_i}_{101\times 15} \longrightarrow \underbrace{\mathbf{x}_i}_{765\text{ powers}} \longrightarrow \underbrace{\mathbf{x}_i'}_{\text{training log/scale transform}} \longrightarrow \underbrace{\mathbf{z}_i}_{5\text{ PCA scores}} \longrightarrow \underbrace{q_i = 0.8}_{\text{calibrated rock probability}} .$$

The actual logarithm must precede scaling; the label under the transform summarizes those saved preprocessing operations. The PCA basis and calibrator are fixed before this packet is evaluated.

Generate visual training targets by repeating the complete tactile pipeline in cross-fitting folds. Associate q_i with the earlier RGB mask of the same physical patch. A visual student with current probability 0.5 has soft-target logit gradient $0.5 - 0.8 = -0.3$, before any confidence weight, so its next update increases the rock score for that appearance. Repeat across many interactions to learn a visual rule.

Evaluate the tactile model against manual labels on untouched preparations. Evaluate the visual model against manual image/patch labels from equally untouched scenes, rather than only against teacher outputs. Compare it with the direct human-label visual baseline. Then separately test changed lighting, speed, moisture, and terrain placement. These tests distinguish visual generalization, teacher quality, and operating-condition robustness.

17 Exercises and selected answers

17.1 Short exercises

1. A dataset has 200 examples and 12 features. Write the shapes of X, $\mathbf{w}$, and $X\mathbf{w}$. Contrast this with one 101×15 telemetry window.

2. A rock example has score $a = -1$. Calculate its sigmoid probability, cross-entropy, and derivative $d\ell/da$. Explain the direction of the next gradient update.

3. A constant channel has value 3 at all times. What are its mean, standard deviation, and RMS? What happens to it after detrending?

4. A real FFT uses 101 samples at 100 Hz. Find its bin spacing, highest stored frequency, number of bins, and flattened dimension across 15 channels. Does zero-padding double its resolving power?

5. For covariance $S = \left[\begin{smallmatrix} 4 & 3 \\ 3 & 4 \end{smallmatrix}\right]$, verify the two directions given in the PCA section. Reconstruct $(2, 1)^\top$ using PC1 only. Can PC2 still matter for classification?

6. An unrefreshed scalar memory retains a fraction 0.9 at each step. What remains after ten steps? How would retention change if a learned gate began replacing most of the state after a terrain transition?

7. A tactile target is $q = 0.6$, while a visual student predicts $p = 0.8$. Find the unweighted soft-target logit gradient. Why is the preferred target different from a hard rock label?

8. Two classifiers achieve 98% and 85% accuracy. The first is evaluated on randomly split overlapping packets; the second on new terrain preparations. What comparison is justified?

17.2 Answers and interpretation

1. The shapes are 200×12, 12×1, and 200×1. Each row of X describes a different example. Each row of a telemetry window instead describes a different time within one example.

2. $p = 1/(1 + e) \approx 0.269$, $\ell = -\log p \approx 1.313$, and $d\ell/da = p - 1 \approx -0.731$. Subtracting a negative score gradient raises the rock score. Individual weight changes also depend on the signs of the features.

3. The mean is 3, the standard deviation is zero, and RMS is 3. Linear detrending removes the constant. A PSD of detrended data cannot preserve this original baseline load; save the mean separately.

4. Spacing is $100/101 \approx 0.9901$ Hz, the highest bin is $50(100/101) \approx 49.505$ Hz, the bin count is 51, and $D = 765$. Zero-padding creates more evaluated frequency points, not a longer observation or new ability to separate nearby sources.

5. The sum and difference directions have eigenvalues 7 and 1. The one-component reconstruction is $(1.5, 1.5)^\top$. PC2 can distinguish terrain even with smaller variance; labels never entered the PCA optimization.

6. $0.9^{10} \approx 0.349$ remains. A strong replacement gate reduces persistence of the old state, helping a model avoid mixing terrain regimes, provided training has learned a useful boundary response.

7. The gradient is $p - q = 0.2$, so training decreases the student's rock score. A hard label 1 would instead give -0.2, increasing it and erasing the teacher's doubt.

8. The scores answer different generalization questions. They do not establish that the first model is better. Evaluate both using the same independently grouped protocol before comparing algorithms.

17.3 Acronyms and further reading

Acronym	Meaning
IMU	Inertial measurement unit.
RGB / PSD	Red–green–blue image / power spectral density.
DFT / FFT / STFT	Discrete / fast / short-time Fourier transform.
RMS / PCA / SVD	Root mean square / principal component analysis / singular value decomposition.
LDA / SVM / RBF	Linear discriminant analysis / support vector machine / radial basis function.
MLP / CNN / TCN	Multilayer perceptron / convolutional neural network / temporal convolutional network.
RNN / LSTM / GRU	Recurrent neural network / long short-term memory / gated recurrent unit.
SSM / SSD	State-space model / state-space duality.
BCE / NLL / KL	Binary cross-entropy / negative log-likelihood / Kullback–Leibler divergence.
ROC / AUC / EMA	Receiver operating characteristic / area under the curve / exponential moving average.

For a broader first course, the classification, resampling, regularization, trees, SVM, deep-learning, and unsupervised-learning chapters of *An Introduction to Statistical Learning* provide a suitable companion [1]. Read the terrain papers after understanding what their inputs, targets, splits, and loss functions mean. Those four questions usually reveal more about a study than its architecture name.

18 Central lessons

1. The repository implements nominal tactile-to-RGB association; physical validation of projection constants and clock alignment is needed to establish which earlier patch matches each tactile event.
2. Sandy/rocky classification and loose/sturdy estimation are related but not identical.
3. A grouped logistic or SVM baseline is scientifically more informative than immediately adopting Mamba.
4. Effective dataset size is the number of independent terrain setups, not the number of overlapping packets.
5. PCA finds variance, not terrain relevance; combine it with supervised validation and ablation.
6. Longer FFT windows improve bin spacing but weaken temporal and spatial localization.
7. Calibrated soft tactile targets preserve ambiguity; compare them with hard pseudo-labels, and validate any additional confidence weighting.
8. *Where Should I Walk?* used ERFNet, Mean Teacher, wavelets, and PCA rather than Mamba.
9. BorealTC is the more relevant Mamba comparison and suggests that Mamba benefits from larger, more diverse data rather than automatically outperforming a spectrogram CNN.
10. Projection constants and cross-stream clock alignment should be verified before a large visual-learning campaign.

References

[1] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: With Applications in Python*. Springer, 2023. Author-provided resources and book: `https://www.statlearning.com/`.

[2] L. Wellhausen, A. Dosovitskiy, R. Ranftl, K. Walas, C. Cadena, and M. Hutter, "Where Should I Walk? Predicting Terrain Properties from Images via Self-Supervised Learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1509–1516, 2019. doi: 10.1109/LRA.2019.2895390.

[3] E. Romera, J. M. Alvarez, L. M. Bergasa, and R. Arroyo, "ERFNet: Efficient Residual Factorized ConvNet for Real-Time Semantic Segmentation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 263–272, 2018. doi: 10.1109/TITS.2017.2750080.

[4] A. Tarvainen and H. Valpola, "Mean Teachers Are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results," in *Advances in Neural Information Processing Systems*, 2017. `https://arxiv.org/abs/1703.01780`.

[5] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," in *Conference on Language Modeling (COLM)*, 2024; original preprint arXiv:2312.00752, December 2023. `https://arxiv.org/abs/2312.00752`.

[6] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," in *Proceedings of the 41st International Conference on Machine Learning*, PMLR vol. 235, pp. 10041–10071, 2024. `https://arxiv.org/abs/2405.21060`.

[7] D. LaRocque, W. Guimont-Martin, D.-A. Duclos, P. Giguère, and F. Pomerleau, "Proprioception Is All You Need: Terrain Classification for Boreal Forests," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11686–11693, 2024. doi: 10.1109/IROS58592.2024.10801407.

[8] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model," arXiv:2401.09417, 2024. `https://arxiv.org/abs/2401.09417`.

[9] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, "VMamba: Visual State Space Model," arXiv:2401.10166, 2024. `https://arxiv.org/abs/2401.10166`.